

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1



BÁO CÁO BÀI TẬP LỚN
PYTHON

Giảng viên hướng dẫn: Kim Ngọc Bách

Sinh viên: Trịnh Lâm Huy-B24DCCE134
Trần Quang Hưng-B24DCCE120

Lớp: D24CQCE01-B

Hà Nội – 2026

MỤC LỤC

LỜI MỞ ĐẦU	1
DANH MỤC THỐNG KÊ	2
CÂU 1	4
CÂU 2	12
CÂU 3	18
CÂU 4	30
CÂU 5	41

LỜI MỞ ĐẦU

Em thực hiện báo cáo này nhằm hoàn thành Bài tập lớn của học phần Lập trình Python. Mục tiêu chính của bài tập là áp dụng các kiến thức về lập trình và xử lý dữ liệu để xây dựng một hệ thống hoàn chỉnh giúp thu thập, phân tích và trực quan hóa dữ liệu thống kê của các cầu thủ thi đấu tại giải **Premier League** mùa giải 2025–2026.

Cụ thể, em đã phát triển một chương trình Python tự động cào dữ liệu từ trang **FBref** đối với các cầu thủ có thời gian thi đấu trên 90 phút. Dữ liệu thu được được sử dụng để xây dựng hệ thống **RESTful API** bằng **Flask** và ứng dụng giao diện **GUI (Tkinter)**, cho phép người dùng tra cứu và so sánh thông số cầu thủ trực quan qua biểu đồ **Radar**. Bên cạnh đó, em cũng áp dụng toán thống kê để phân tích, chấm điểm xếp hạng phong độ đội bóng, đồng thời sử dụng thuật toán **K-means** kết hợp **PCA** để phân cụm và trực quan hóa vai trò chiến thuật của các cầu thủ trên sân.

Báo cáo này trình bày chi tiết quá trình thực hiện, các phương pháp giải quyết vấn đề, kết quả trực quan hóa, cũng như những nhận xét, đánh giá cá nhân trong suốt quá trình hoàn thiện bài tập.

Em xin chân thành cảm ơn!

DANH MỤC CÁC CHỈ SỐ THỐNG KÊ

Các chỉ số trong bộ dữ liệu được thu thập từ hệ thống FBref, bao gồm thông tin cơ bản và các thông số chuyên môn chi tiết. Để thuận tiện cho việc theo dõi, các chỉ số được phân thành các nhóm chức năng chính như sau:

1. Nhóm Thông tin Cơ bản (General Information)

Ký hiệu	Tên đầy đủ	Giải nghĩa
Player	Player Name	Tên của cầu thủ.
Nation	Nationality	Quốc tịch của cầu thủ.
Pos	Position	Vị trí thi đấu
Squad	Team / Club	Tên câu lạc bộ chủ quản.
Age	Age	Độ tuổi tính đến thời điểm thu thập dữ liệu.

2. Nhóm Thời gian thi đấu (Playing Time)

Ký hiệu	Tên đầy đủ	Giải nghĩa
MP	Matches Played	Tổng số trận đã tham gia.
Starts	Games Started	Số trận ra sân trong đội hình xuất phát.
Min	Minutes Played	Tổng số phút thi đấu thực tế.
90s	90 Minutes Played	Số phút thi đấu quy đổi ra đơn vị 90 phút (Min/90).
Mn/MP	Minutes per Match	Số phút thi đấu trung bình mỗi trận.

3. Nhóm Hiệu suất Tấn công (Attacking Performance)

Ký hiệu	Tên đầy đủ	Giải nghĩa
Gls	Goals	Tổng số bàn thắng ghi được.
Ast	Assists	Tổng số đường kiến tạo thành bàn.
Sh / Sh/90	Shots	Tổng số cú sút / Số cú sút trung bình mỗi 90 phút.
SoT / SoT%	Shots on Target	Số cú sút trúng đích / Tỷ lệ sút trúng đích (%).
G/Sh	Goals per Shot	Tỷ lệ bàn thắng trên mỗi cú sút (Hiệu suất dứt điểm).

4. Nhóm Phòng ngự và Kỷ luật (Defensive & Discipline)

Ký hiệu	Tên đầy đủ	Giải nghĩa
TklW	Tackles Won	Số pha tắc bóng thành công và giành lại bóng.
Int	Interceptions	Số lần đánh chặn đường chuyền của đối phương.
Crs	Crosses	Số đường tạt bóng từ hai biên.
Fls / Fld	Fouls Committed/Drawn	Số lần phạm lỗi / Số lần bị phạm lỗi.
CrdY / CrdR	Yellow / Red Cards	Số thẻ vàng và thẻ đỏ phải nhận.

5. Nhóm Chỉ số Thủ môn (Goalkeeping - Dành riêng cho GK)

Ký hiệu	Tên đầy đủ	Giải nghĩa
GA / GA90	Goals Against	Tổng số bàn thua / Số bàn thua trung bình mỗi 90 phút.
Saves	Saves	Số pha cứu thua thành công.
Save%	Save Percentage	Tỉ lệ cứu thua thành công (%).
CS / CS%	Clean Sheets	Số trận giữ sạch lưới / Tỉ lệ giữ sạch lưới (%).

6. Nhóm Chỉ số Đóng góp vào lối chơi chung (Team Success)

Ký hiệu	Tên đầy đủ	Giải nghĩa
PPM	Points Per Match	Điểm số trung bình đội bóng đạt được khi cầu thủ thi đấu.
+/-	Plus/Minus	Hiệu số bàn thắng-bàn thua của đội khi cầu thủ có mặt.
+/-90	Plus/Minus per 90m	Hiệu số bàn thắng-bàn thua trung bình mỗi 90 phút.

CÂU 1: Bài tập này được bọn em lưu code vào file **crawling.py**

1.1. Ý tưởng chung

Để giải quyết bài toán thu thập dữ liệu từ FBref, chương trình sử dụng trình duyệt ẩn danh để truy cập lần lượt vào danh sách các đường link thống kê. Khi mã nguồn trang web được tải về, thư viện **BeautifulSoup** sẽ quét tìm chính xác bảng dữ liệu của cầu thủ. Điểm mấu chốt của thuật toán bóc tách là việc xử lý cấu trúc tiêu đề bảng 2 tầng bằng cách dựa vào thuộc tính **colspan** ở dòng trên để ghép nối với tên chỉ số ở dòng dưới, từ đó tạo ra một bộ từ điển ánh xạ tên cột đầy đủ và chi tiết nhất. Sau đó, dữ liệu thô của các cầu thủ (đáp ứng điều kiện thi đấu trên 90 phút) sẽ được trích xuất, đưa vào **Pandas** để hợp nhất, đổi tên cột theo từ điển và xuất ra định dạng **CSV** chuẩn hóa.

Trong quá trình crawl dữ liệu khi gặp các tình huống như Captcha và hoặc rate-limit, bọn em có đề xuất các phương pháp khắc phục như sau:

- **Khắc phục Rate-limit (Bị chặn do thao tác quá nhanh):** Tạo hàm trễ ngẫu nhiên (random_delay) từ 3 đến 7 giây giữa mỗi lần chuyển trang để giả lập tốc độ đọc web của người thật.
- **Khắc phục CAPTCHA / Anti-Bot:** Không dùng Selenium thường, dùng Undetected ChromeDriver (UC Mode) của **SeleniumBase** để tàng hình trước hệ thống phát hiện Bot của Cloudflare. Tích hợp lệnh sb.uc_gui_click_captcha() để tool tự động nhận diện và click vào ô "I am human" nếu bị trang web kiểm tra.

1.2. Triển khai bài toán

- Khai báo thư viện

```
import pandas as pd
import random
import time
from bs4 import BeautifulSoup as bs
from seleniumbase import SB
import os
```

- **pandas:** Thư viện lõi dùng để tạo, xử lý và lưu trữ dữ liệu dưới dạng bảng (DataFrame).

- **random, time:** Cung cấp các hàm tạo thời gian trễ (delay) ngẫu nhiên, giúp hành vi của chương trình giống với người dùng thật.
- **BeautifulSoup:** Thư viện phân tích cú pháp HTML, giúp tìm kiếm và trích xuất dữ liệu từ các thẻ (tags) của trang web một cách dễ dàng.
- **seleniumbase (SB):** Công cụ tự động hóa trình duyệt web. Khác với Selenium thông thường, SeleniumBase hỗ trợ chế độ UC (Undetected) giúp vượt qua các bài kiểm tra Captcha.
- **os:** Cung cấp các hàm làm việc với hệ thống tệp, đặc biệt dùng để xử lý đường dẫn file trong chương trình.

- Khởi tạo các đường dẫn truy cập đến thư mục và file

```
BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
OUTPUT_PATH = os.path.join(BASE_DIR, 'output')
file_path = os.path.join(OUTPUT_PATH, 'data.csv')
```

- Hàm tạo độ trễ

```
def random_delay(min_seconds=5, max_seconds=20):
    time.sleep(random.uniform(min_seconds, max_seconds))
```

- Khi crawl web liên tục, nếu request quá nhanh và đều đặn, server sẽ nhận diện là Bot và khóa IP. Vậy nên, hàm này sử dụng **random.uniform()** để bắt chương trình tạm dừng ngẫu nhiên từ 5 đến 15 giây trước mỗi lần tải trang mới, đảm bảo an toàn cho quá trình cào dữ liệu.

- Khởi tạo danh sách các mã định danh các cột, url cần thiết và các biến lưu trữ

```
all_cols = ['player', 'nationality', 'position', 'team', 'age', 'games', 'games_starts', 'minutes', 'minutes_90s',
            'goals', 'assists', 'goals_assists', 'goals_pens', 'pens_made', 'pens_att', 'cards_yellow', 'cards_red',
            'goals_per90', 'assists_per90', 'goals_assists_per90', 'goals_pens_per90', 'goals_assists_pens_per90',
            'gk_goals_against', 'gk_goals_against_per90', 'gk_shots_on_target_against', 'gk_saves', 'gk_save_pct',
            'gk_wins', 'gk_ties', 'gk_losses', 'gk_clean_sheets', 'gk_clean_sheets_pct', 'gk_pens_att',
            'gk_pens_allowed', 'gk_pens_saved', 'gk_pens_missed', 'gk_pens_save_pct', 'shots', 'shots_on_target',
            'shots_on_target_pct', 'shots_per90', 'shots_on_target_per90', 'goals_per_shot',
            'goals_per_shot_on_target', 'minutes_per_game', 'minutes_pct', 'minutes_per_start', 'games_complete',
            'games_subs', 'minutes_per_sub', 'unused_subs', 'points_per_game', 'on_goals_for', 'on_goals_against',
            'plus_minus', 'plus_minus_per90', 'plus_minus_wowwy', 'cards_yellow_red', 'fouls', 'fouled', 'offsides',
            'crosses', 'interceptions', 'tackles_won', 'pens_won', 'pens_conceded', 'own_goals']
```

- Là danh sách chứa các mã định danh tương ứng tuyệt đối với thuộc tính ‘data-stat’ trong mã HTML của trang FBref. Ví

```
<th aria-label="Ranker" data-stat="ranker" scope="col">
<th aria-label="Player" data-stat="player" scope="col">
<th aria-label="Nation" data-stat="nationality" scope="col">
<th aria-label="Position" data-stat="position" scope="col">
<th aria-label="Squad" data-stat="team" scope="col">
<th aria-label="Current age" data-stat="age" scope="col">
```

dụ:

- Giúp hàm trích xuất định vị chính xác ô cần lấy số liệu, đảm bảo tool hoạt động ổn định và không bị lỗi ngay cả khi trang web thay đổi giao diện hiển thị.

```
all_url = ['https://fbref.com/en/comps/9/stats/Premier-League-Stats',
           'https://fbref.com/en/comps/9/keepers/Premier-League-Stats',
           'https://fbref.com/en/comps/9/shooting/Premier-League-Stats',
           'https://fbref.com/en/comps/9/playingtime/Premier-League-Stats',
           'https://fbref.com/en/comps/9/misc/Premier-League-Stats']

auto_rename_map = {}
rows = []
```

- **all_url**: Danh sách toàn bộ các đường link dẫn đến các bảng thống kê chi tiết chuẩn bị cào.
- **auto_rename_map = {}**: Khởi tạo một từ điển (dictionary) rỗng dùng để lưu các quy tắc ghép/đổi tên cột (từ mã gốc trên web sang tên hiển thị chuẩn) hỗ trợ cho việc làm sạch dữ liệu sau này.
Ví dụ: ‘90s Played’ (data-stat) -> ‘Playing Time 90s’ (display_name)
- **rows = []**: tạo một danh sách (list) rỗng, là nơi chứa tạm thời toàn bộ dữ liệu thống kê đã trích xuất của từng cầu thủ trước khi được gộp và chuyển đổi thành dạng bảng (Pandas DataFrame) ở cuối chương trình.

- Hàm trích xuất dữ liệu từng ô


```
def get(player, stat, extra=False):
    tmp = player.find('td', attrs={'data-stat': stat})
    try:
        tmp = tmp.text
        if extra:
            tmp = tmp.split()[1]
        tmp = tmp.strip()
        if not tmp:
            return None
        return tmp
    except:
        return None
```

- Hàm **get** được dùng để lấy giá trị text của một chỉ số thống kê cụ thể từ thẻ **<td>** tương ứng.
- Tham số **extra=False** đóng vai trò là một cờ (flag). Nếu **extra=True** (thường áp dụng cho cột Quốc tịch), chương trình sẽ sử dụng **split()[1]** để cắt bỏ chuỗi thừa (ví dụ: chuyển "eng ENG" thành "ENG").
- Xử lý ngoại lệ (**try...except**): Nếu không tìm thấy thẻ hoặc giá trị bị rỗng, hàm sẽ trả về **None**.

- Khởi tạo trình duyệt ẩn danh

```
with SB(uc=True) as sb:
```

- **with SB(uc=True) as sb:** Khởi động trình duyệt ẩn danh (giúp tool không bị web nhận diện là Bot) và tự động đóng khi chạy xong.

- Sau đó, chọn từng url trong danh sách để truy cập và cào dữ liệu

```
for url in all_url:

    sb.uc_open_with_reconnect(url)
    sb.uc_gui_click_captcha()
    random_delay(3, 7)
```

- **sb.uc_open_with_reconnect(url):** Truy cập trang web với cơ chế tự động kết nối lại để lách tường lửa (như Cloudflare).
- **sb.uc_gui_click_captcha():** Tự động click xác nhận "Tôi không phải robot" nếu trang web yêu cầu.
- **random_delay(3, 7):** Tạm dừng ngẫu nhiên 3 đến 7 giây để giả lập độ trễ giống người thật, giúp tránh bị khóa IP.

- Lấy và phân tích mã nguồn HTML

```
html = sb.get_page_source()
soup = bs(html, 'html.parser')
```

- Tìm bảng chứa tên, chỉ số cầu thủ trong mỗi link URL

```
tables = soup.find_all('table')
player_table = None
for tbl in tables:
    if tbl.find('th', attrs={'data-stat': 'player'}):
        player_table = tbl
        break

if not player_table:
    continue
```

- Trong mã nguồn HTML, tìm tất cả thẻ bảng **<table>** qua câu lệnh **tables = soup.find_all('table')**
- Duyệt các thẻ bảng vừa tìm được, ta lọc ra bảng có thuộc tính **'data-stat': 'player'** và thường thì trong các trang của Fbref thì thông tin chỉ số của các cầu thủ chỉ nằm trong một thẻ **<table>** duy nhất nên nếu tìm được ta có thể **break** (ko cần thiết phải duyệt thêm các thẻ bảng còn lại)

- Lọc các cầu thủ có số phút thi đấu nhiều hơn 90 phút

```
players = player_table.find_all('tr')
for player in players:
    try:
        time_val = player.find('td', attrs={'data-stat': 'minutes_90s'}).text.split('.')
        if int(time_val[0]) >= 1:
            rows.append({i: get(player, i, True if i == 'nationality' else False) for i in all_cols})
    except:
        continue
```

- **Duyệt danh sách cầu thủ:** Lệnh `player_table.find_all('tr')` quét qua từng dòng (tương ứng với một cầu thủ) trong bảng dữ liệu.
- **`player.find('td', attrs={'data-stat': 'minutes_90s'}).text.split('.')`:** Chương trình tìm cột thời gian thi đấu '`minutes_90s`', cắt lấy phần nguyên và lọc xem chỉ số có ≥ 1 hay không.
- Nếu cầu thủ thỏa mãn điều kiện, một vòng lặp thu gọn (List Comprehension) sẽ gọi hàm `get()` quét qua toàn bộ mã chỉ số trong `all_cols` để tạo thành một dòng dữ liệu (dictionary) hoàn chỉnh và lưu vào mảng `rows`. Cờ `True` được bật riêng cho cột `nationality` để cắt bỏ các chuỗi thừa.

- Xử lý tiêu đề bảng (Header) và chuẩn hóa tên cột

```

table_head = player_table.find('thead')
if table_head:
    rows_head = table_head.find_all('tr')
    if len(rows_head) == 1:
        for th in rows_head[0].find_all('th', attrs={'data-stat': True}):
            stat_code = th.get('data-stat')
            display_name = th.text.strip()
            if stat_code not in auto_rename_map or len(display_name) > len(auto_rename_map[stat_code]):
                auto_rename_map[stat_code] = display_name

    elif len(rows_head) >= 2:
        top_row = rows_head[0].find_all('th')
        bottom_row = rows_head[1].find_all('th')
        bottom_idx = 0

        for th in top_row:
            prefix = th.text.strip()
            colspan = int(th.get('colspan', 1))

```

```

for th in top_row:
    prefix = th.text.strip()
    colspan = int(th.get('colspan', 1))

    for i in range(colspan):
        if bottom_idx < len(bottom_row):
            bot_th = bottom_row[bottom_idx]
            bot_stat = bot_th.get('data-stat')
            bot_text = bot_th.text.strip()

            if bot_stat:
                full_name = f"{prefix} {bot_text}".strip() if prefix else bot_text

                if bot_stat not in auto_rename_map:
                    auto_rename_map[bot_stat] = full_name
                elif len(full_name) > len(auto_rename_map[bot_stat]):
                    auto_rename_map[bot_stat] = full_name

            bottom_idx += 1

```

- Bảng 1 dòng tiêu đề: Nếu bảng chỉ có 1 dòng tiêu đề (phát hiện bằng **len(rows_head) == 1**), chương trình đơn giản quét qua các thẻ **<th>**, lấy mã định danh (**data-stat**) làm key và tên hiển thị làm value để đưa vào từ điển **auto_rename_map**.
- Bảng nhiều dòng tiêu đề (Gộp cột kép): Trên trang FBref, các bảng chi tiết thường dùng 2 dòng tiêu đề gộp nhau (bằng colspan). Nếu bảng có từ 2 dòng trở lên (**len(rows_head) >= 2**), đoạn code trên sẽ đọc dòng trên cùng (**top_row**) lấy làm "tiền tố" (**prefix**). Dựa vào độ rộng colspan, nó đếm số lượng cột con tương ứng ở dòng dưới (**bottom_row**) và ghép tiền tố này với tên cột con (Ví dụ: "Performance" + "Goals" = "Performance Goals").

	Playing Time					Performance								Per 90 Minutes				
3orn	MP	Starts	Min	90s	Gls	Ast	G+A	G-PK	PK	PKatt	CrdY	CrdR	Gls	Ast	G+A	G-PK	G+A-PK	

- Cập nhật từ điển thông minh: Dù là bảng 1 dòng hay 2 dòng, chương trình luôn kiểm tra xem tên cột mới ghép được có dài và đầy đủ nghĩa hơn tên đã lưu trước đó trong **auto_rename_map** hay không. Nếu có, nó sẽ ghi đè để ưu tiên tên chuẩn nhất.

- Gộp dữ liệu, làm sạch và xuất file

```

if rows:
    data_all = pd.DataFrame(rows, columns=all_cols)
    data_all = data_all.groupby('player').first().reset_index()

    if auto_rename_map:
        data_all = data_all.rename(columns=auto_rename_map)

    data_all.fillna("N/a").to_csv('output/data.csv', index=False, encoding='utf-8-sig')
    print('DONE')
else:
    print('No data collected')

```

- Chuyển đổi và Gộp dòng: Chương trình chuyển mảng **rows** thành **Pandas DataFrame**. Do dữ liệu được cào từ nhiều bảng khác nhau, một cầu thủ sẽ bị lặp lại nhiều dòng. Lệnh **groupby('player').first()** giúp gộp toàn bộ các chỉ số của cùng một cầu thủ về chung một dòng duy nhất.
- Đổi tên cột: Lệnh **rename(columns=auto_rename_map)** áp dụng bộ từ điển đã xây dựng ở bước trước để dịch các mã định danh (như **goals_per90**) thành tên cột hoàn chỉnh, đúng như hiển thị trên Web.
- Lưu trữ dữ liệu: Lệnh **fillna("N/a")** lấp đầy các ô trống thiếu dữ liệu. Cuối cùng, hàm **to_csv** xuất toàn bộ kết quả ra file **data.csv** với chuẩn mã hóa **utf-8-sig** để đảm bảo không bị lỗi font chữ khi mở bằng Excel.

- Kết quả 1 phần của file data.csv

Player	Nation	Pos	Squad	Age	Playing Time MP	Playing Time Starts	Playing Time Min	Playing Time 90s	Performance Gls	Performance Ast
1 Aaron Hickey	SCO	DF	Brentford	23-319	18	7	599	6.7	0	
2 Aaron Ramsdale	ENG	GK	Newcastle United	27-346	12	11	1,004	11.2	0	
3 Aaron Wan-Bissaka	COD	DF,MF	West Ham United	28-150	22	21	1,903	21.1	0	
4 Abdulkodir Khusanov	UZB	DF	Manchester City	22-055	18	12	1,162	12.9	0	
5 Adam Armstrong	ENG	FW,MF	Wolves	29-074	9	9	726	8.1	1	
6 Adam Smith	ENG	DF	Bournemouth	34-361	19	11	809	9.0	0	
7 Adam Wharton	ENG	MF	Crystal Palace	22-078	28	26	2,190	24.3	0	
8 Adama Traoré	ESP	MF	Fulham	30-090	15	1	274	3.0	0	
9 Adrien Truffert	FRA	DF	Bournemouth	24-156	34	34	3,019	33.5	1	
10 Alejandro Garnacho	ARG	MF	Chelsea	21-298	23	14	1,266	14.1	1	
11 Alex Iwobi	NGA	MF	Fulham	29-357	27	27	2,263	25.1	4	
12 Alex Scott	ENG	MF	Bournemouth	22-247	34	31	2,580	28.7	3	
13 Alexander Isak	SWE	FW	Liverpool	26-216	12	7	597	6.6	2	
14 Alexis Mac Allister	ARG	MF	Liverpool	27-122	32	26	2,208	24.5	2	
15 Alisson	BRA	GK	Liverpool	33-205	25	25	2,250	25.0	0	
16 Alphonse Areola	FRA	GK	West Ham United	33-057	20	20	1,800	20.0	0	
17 Altay Bayindir	TUR	GK	Manchester Utd	28-011	6	6	540	6.0	0	
18 Amad Diallo	CIV	MF	Manchester Utd	23-288	27	23	1,997	22.2	2	
19 Amadou Onana	BEL	MF	Aston Villa	24-252	24	21	1,740	19.3	2	
20 Amlin Adli	MAR	MF	Bournemouth	25-350	28	10	1,058	11.8	3	
21 Andrew Robertson	SCO	DF	Liverpool	32-045	21	8	950	10.6	0	
22 Andrey Santos	BRA	MF	Chelsea	21-357	24	11	1,075	11.9	0	
23 André	BRA	MF	Wolves	24-283	30	25	2,312	25.7	1	
24 Andy Irving	SCO	MF	West Ham United	25-347	7	2	158	1.8	0	
25 Angel Gomes	ENG	MF,FW	Wolves	25-237	9	6	381	4.2	0	
26 Anthony Elanga	SWE	FW,MF	Newcastle United	23-363	29	14	1,283	14.3	0	
27 Anthony Gordon	ENG	FW	Newcastle United	25-060	26	24	1,795	19.9	6	
28 Antoine Semenyo	GHA	MF	Bournemouth	26-108	20	20	1,798	20.0	10	
29 Anton Stach	GER	MF	Leeds United	27-161	26	25	2,136	23.7	4	
30 Antonee Robinson	USA	DF,MF	Fulham	28-260	17	13	1,122	12.5	0	

CÂU 2: Bài tập này được bạn em lưu code vào file **api.py**

2.1. Ý tưởng chung

Để giải quyết bài toán phân phối và truy xuất thông tin, chương trình của bạn em sử dụng framework Flask để xây dựng một RESTful API. Đầu tiên, hệ thống được thiết kế cơ chế tự động dò tìm và nạp file data.csv (dữ liệu đã làm sạch ở bài 1) vào bộ nhớ thông qua thư viện Pandas. Để tối ưu trải nghiệm truy vấn, API hỗ trợ tìm kiếm cầu thủ qua hai phương thức định tuyến linh hoạt: truyền tham số qua URL (Query String) hoặc truyền trực tiếp vào đường dẫn (Path Variable). Khi nhận yêu cầu, thuật toán sẽ thực hiện tìm kiếm chuỗi con (không phân biệt chữ hoa, chữ thường) trên tập dữ liệu. Nếu tìm thấy kết quả khớp, toàn bộ chỉ số thống kê của cầu thủ sẽ lập tức được đóng gói và trả về dưới định dạng JSON chuẩn hóa.

2.2. Triển khai

- Khai báo thư viện

```
from flask import Flask, request, jsonify
import pandas as pd
import os
```

- **Flask:** Là framework chính dùng để xây dựng RESTful API, giúp tạo server và định nghĩa các endpoint xử lý request từ client.
- **request:** Được sử dụng để lấy dữ liệu gửi từ phía client (ví dụ: query string, tham số URL).
- **jsonify:** Dùng để chuyển đổi dữ liệu Python (dict, list) sang định dạng JSON để trả về cho client.
- **pandas:** Thư viện lõi dùng để tạo, xử lý và lưu trữ dữ liệu dưới dạng bảng (DataFrame).
- **os:** Cung cấp các hàm làm việc với hệ thống tệp, đặc biệt dùng để xử lý đường dẫn file trong chương trình.

- Khởi tạo ứng dụng Flask


```
app = Flask(__name__)
```

- Cấu hình hiển thị dữ liệu JSON

```
app.json.ensure_ascii = False
```

- Thiết lập cấu hình cho Flask để đảm bảo dữ liệu JSON trả về giữ nguyên các ký tự Unicode. Ví dụ dữ liệu sẽ trả về 'Rúben Dias' thay vì 'R\u00faben Dias',

- **Xác định thư mục gốc của project:** Lấy đường dẫn tuyệt đối của file hiện tại, đi lùi 2 cấp thư mục để xác định thư mục gốc của project.

```
_PROJECT_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
```

- **Hàm tìm file CSV:** Kiểm tra lần lượt các vị trí có thể chứa file **data.csv** Ưu tiên thư mục gốc, sau đó đến thư mục **output/**. Nếu tìm thấy trả về đường dẫn, ngược lại trả về **None**.

```
def find_csv():  
    for candidate in [  
        os.path.join(_PROJECT_ROOT, "data.csv"),  
        os.path.join(_PROJECT_ROOT, "output", "data.csv"),  
    ]:  
        if os.path.exists(candidate):  
            return candidate  
    return None
```

- Hàm đọc dữ liệu

```

dataframe = None
def get_data():
    global dataframe
    if dataframe is None:
        file_path = find_csv()
        if not file_path:
            return None
        try:
            dataframe = pd.read_csv(file_path)
        except Exception as e:
            print("Error loading CSV:", e)
            dataframe = None
    return dataframe

```

- Khởi tạo biến toàn cục **dataframe = None** dùng để lưu trữ dữ liệu file CSV, đồng thời tránh việc đọc lại file nhiều lần khi có nhiều request.
- **Lượt truy cập đầu tiên:** Khi biến **dataframe is None**, hệ thống mới thực hiện đọc file **data.csv** và nạp dữ liệu vào biến.
- **Các lượt truy cập sau:** Hệ thống bỏ qua hoàn toàn việc đọc ổ cứng và trả về trực tiếp dữ liệu đã được lưu sẵn trong biến **dataframe**.
- **Lợi ích:** Tránh việc đọc file liên tục ở mỗi lần tìm thông tin, giúp API phản hồi cực nhanh và tránh quá tải server khi có nhiều lượt truy cập cùng lúc.

- Hàm tìm kiếm cầu thủ


```
def search_player(name):
    df = get_data()

    if df is None:
        return jsonify({'error': 'Data file not found'}), 500

    if 'Player' not in df.columns:
        return jsonify({'error': 'Invalid data format'}), 500

    player_data = df[df['Player'].str.contains(name, case=False, na=False)]

    if player_data.empty:
        return jsonify({'error': 'Player data not found'}), 404

    data = player_data.to_dict(orient='records')
    return jsonify({'data': data}), 200
```

- Đầu tiên, chương trình gọi hàm **get_data()** để lấy dữ liệu phục vụ tìm kiếm. Câu lệnh **if df is None** kiểm tra quá trình nạp file ở bước trước thất bại, hệ thống sẽ lập tức dừng xử lý và trả về mã lỗi **500 (Internal Server Error)**.
- Kiểm tra cấu trúc dữ liệu: hệ thống sẽ quét cấu trúc của **DataFrame** để đảm bảo cột **'Player'** thực sự tồn tại. Nếu file CSV bị hỏng cấu trúc hoặc sai định dạng, API sẽ từ chối xử lý và báo lỗi **500**.
- **df[df['Player'].str.contains(name, case=False, na=False)]**: Thực hiện tìm kiếm theo chuỗi con (**str.contains(name)**), cùng với đó là không phân biệt chữ hoa/thường (**case = False**). Người dùng không cần gõ chính xác 100% tên cầu thủ. Ví dụ chỉ cần gõ "Haal" thay vì "Erling Haaland".
- Tiếp theo, thuật toán tiến hành kiểm tra nếu DataFrame rỗng, API sẽ trả về mã **404 (Not Found)**. Ngược lại, nếu tìm thấy dữ liệu, hàm **to_dict(orient='records')** được sử dụng để chuyển đổi cấu trúc bảng của Pandas thành định dạng Dictionary. Cuối cùng, hàm **jsonify** đóng gói dữ liệu thành chuẩn JSON phổ quát với mã trạng thái **200 (OK)**.

- Xây dựng API tra cứu bằng Query String (/api/player?name=Haaland)

```
# Tra cứu bằng query string: /api/player?name=Haaland
@app.route('/api/player', methods=['GET'])
def get_player_stats_query():
    name_query = request.args.get('name')
    if not name_query:
        return jsonify({'error': 'Missing name query parameter'}), 400

    return search_player(name_query)
```

- **@app.route('/api/player', methods=['GET']):** Định nghĩa endpoint **/api/player**, chỉ chấp nhận phương thức **GET**.
- **request.args.get('name'):** Lấy giá trị của tham số **'name'** từ URL request.
- Kiểm tra dữ liệu đầu vào: Nếu không có tham số **'name'**, API sẽ trả về lỗi **400 Bad Request** kèm thông báo phù hợp.
- Nếu tham số hợp lệ: Gọi hàm **search_player(name_query)** để thực hiện tìm kiếm dữ liệu trong file CSV. Kết quả được trả về dưới dạng JSON cho client.

- Xây dựng API tra cứu bằng path variable (/api/player/Haaland)

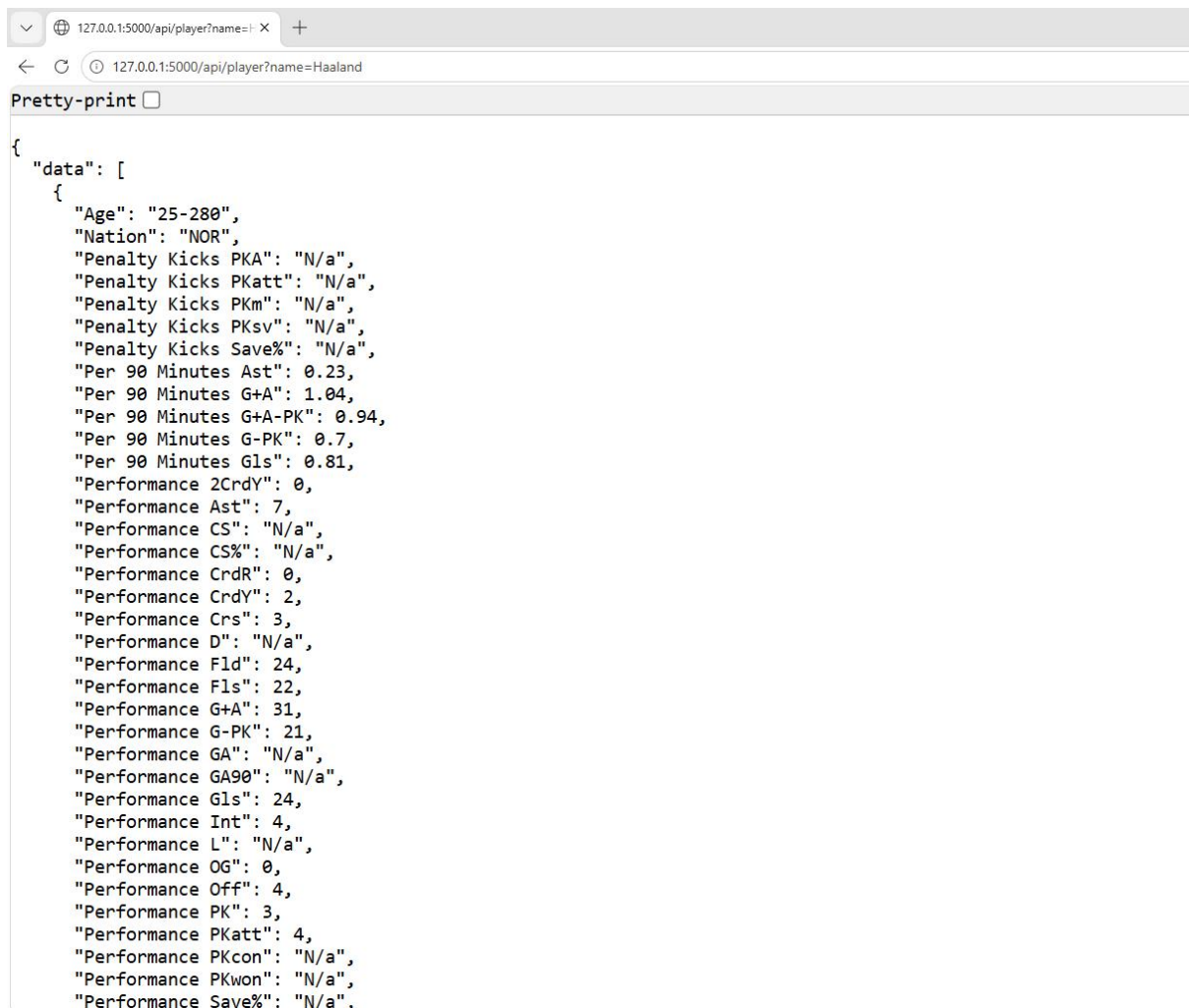
```
# Tra cứu bằng path variable: /api/player/Haaland
@app.route('/api/player/<string:name>', methods=['GET'])
def get_player_stats_path(name):
    return search_player(name)
```

- **@app.route('/api/player/<string:name>', methods=['GET']):** Định nghĩa endpoint với tham số động **'name'** nằm trong đường dẫn URL, chỉ chấp nhận phương thức **GET**.
- Khi client gửi request: **/api/player/Haaland** => Flask sẽ tự động gán giá trị **"Haaland"** cho biến **'name'**.
- Hàm **get_player_stats_path(name)** nhận tham số này và truyền vào hàm **search_player(name)** để thực hiện tìm kiếm dữ liệu. Kết quả được trả về dưới dạng JSON.

- Khởi chạy ứng dụng Flask

```
# Khởi chạy server
if __name__ == '__main__':
    print("Starting Flask server...")
    print("API Endpoints:")
    print("1. GET /api/player?name=<player_name>")
    print("2. GET /api/player/<player_name>")
    app.run(debug=True)
```

- Kết quả



The screenshot shows a web browser with the address bar displaying `127.0.0.1:5000/api/player?name=Haaland`. The page content shows a JSON response from the API, which is pretty-printed. The JSON structure is as follows:

```
{
  "data": [
    {
      "Age": "25-280",
      "Nation": "NOR",
      "Penalty Kicks PKA": "N/a",
      "Penalty Kicks PKatt": "N/a",
      "Penalty Kicks PKm": "N/a",
      "Penalty Kicks PKsv": "N/a",
      "Penalty Kicks Save%": "N/a",
      "Per 90 Minutes Ast": 0.23,
      "Per 90 Minutes G+A": 1.04,
      "Per 90 Minutes G+A-PK": 0.94,
      "Per 90 Minutes G-PK": 0.7,
      "Per 90 Minutes Gls": 0.81,
      "Performance 2CrdY": 0,
      "Performance Ast": 7,
      "Performance CS": "N/a",
      "Performance CS%": "N/a",
      "Performance CrdR": 0,
      "Performance CrdY": 2,
      "Performance Crs": 3,
      "Performance D": "N/a",
      "Performance Fld": 24,
      "Performance Fls": 22,
      "Performance G+A": 31,
      "Performance G-PK": 21,
      "Performance GA": "N/a",
      "Performance GA90": "N/a",
      "Performance Gls": 24,
      "Performance Int": 4,
      "Performance L": "N/a",
      "Performance OG": 0,
      "Performance Off": 4,
      "Performance PK": 3,
      "Performance PKatt": 4,
      "Performance PKcon": "N/a",
      "Performance PKwon": "N/a",
      "Performance Save%": "N/a",
    }
  ]
}
```

CÂU 3: Bài tập này được em lưu vào file **gui.py**

3.1 Ý tưởng chung

Chương trình xây dựng một giao diện người dùng (GUI) nhằm hỗ trợ tìm kiếm và so sánh chỉ số giữa hai cầu thủ bóng đá dựa trên dữ liệu lấy từ API của câu 2 vừa làm. Người dùng nhập tên hai cầu thủ vào hai ô tìm kiếm tương ứng. Sau khi gửi yêu cầu, chương trình sẽ hiển thị các thông tin và chỉ số thống kê của từng cầu thủ. Người dùng có thể lựa chọn các chỉ số mong muốn thông qua các checkbox. Cuối cùng, hệ thống sẽ trực quan hóa kết quả so sánh bằng biểu đồ radar, giúp người dùng dễ dàng nhận biết sự khác biệt giữa hai cầu thủ.

3.2 Triển khai

- Khai báo thư viện

```
import tkinter as tk
from tkinter import messagebox
import requests
import matplotlib.pyplot as plt
import numpy as np
```

- **tkinter**: xây dựng giao diện người dùng
- **messagebox**: hiển thị thông báo lỗi hoặc cảnh báo
- **requests**: gửi yêu cầu HTTP tới API để lấy dữ liệu
- **matplotlib**: vẽ biểu đồ radar
- **numpy**: hỗ trợ tính toán các góc trong biểu đồ

- Cấu trúc chương trình

Chương trình được xây dựng theo mô hình lập trình hướng đối tượng (OOP) thông qua một class chính:

```
class PlayerCompareGUI:
```

Class này chịu trách nhiệm quản lý toàn bộ:

- Giao diện người dùng
- Dữ liệu cầu thủ

- Các chức năng tìm kiếm và so sánh

=> Việc sử dụng OOP giúp chương trình dễ quản lý và tránh sử dụng biến toàn cục.

- Hàm khởi tạo giao diện chương trình

```
def __init__(self, root):
```

- Khởi tạo cửa sổ chính

```
self.root = root
self.root.title('Player Search & Comparison')
self.root.geometry('900x600')
```

- Khởi tạo các biến dữ liệu

```
self.api_url = 'http://127.0.0.1:5000/api/player'
self.data_p1 = None
self.data_p2 = None

# Từ điển để lưu các checkbox được tích chọn
# Chỉ số (stat_name) -> BooleanVar
self.check_vars = {}
```

- Tạo giao diện chính

```
# Giao diện chính chia làm 2 cột
self.frame_main = tk.Frame(root)
self.frame_main.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)
```

- Tạo hai cột hiển thị cầu thủ


```
# Khung chứa cột 1
self.frame_p1 = tk.Frame(self.frame_main, bd=1, relief='ridge')
self.frame_p1.pack(side=tk.LEFT, fill=tk.BOTH, expand=True, padx=5)

# Khung chứa cột 2
self.frame_p2 = tk.Frame(self.frame_main, bd=1, relief='ridge')
self.frame_p2.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True, padx=5)
```

- **Tạo khu vực nhập Player 1 (Tương tự với Player 2)**

```
# ----- CỘT 1 -----
tk.Label(self.frame_p1, text='Player 1', font=('Arial', 12, 'bold')).pack(pady=5)
self.entry_p1 = tk.Entry(self.frame_p1, font=('Arial', 12))
self.entry_p1.pack(fill=tk.X, padx=10, pady=5)
tk.Button(self.frame_p1, text='Search', bg='blue', fg='white', command=lambda: self.search_player(1)).pack(pady=5)

self.canvas_p1, self.scroll_p1, self.inner_p1 = self.create_scrollable_frame(self.frame_p1)
```

- **tk.Label, tk.Entry, tk.Button** và **.pack()**: Các lệnh cơ bản để khởi tạo và bố trí thành phần giao diện (tiêu đề "Player 1", ô nhập văn bản, và nút "Search").
- **command=lambda: self.search_player(1)**: Gán sự kiện cho nút bấm. Khi được nhấn, nó sẽ gọi đến hàm `search_player` (*chi tiết về luồng xử lý API và tìm kiếm của hàm này sẽ được trình bày ở mục sau*).
- **self.create_scrollable_frame(...)**: Gọi hàm phụ trợ (*sẽ được phân tích chi tiết ở phần kế tiếp*) nhằm tạo một khung chứa có thanh cuộn dọc (scrollbar), chuẩn bị không gian để hiển thị danh sách dài các chỉ số thống kê của cầu thủ.

- **Tạo nút Compare**

```
# ----- NÚT SO SÁNH -----
tk.Button(root, text='Compare', font=('Arial', 14, 'bold'), bg='#4CAF50', fg='white', command=self.compare).pack(pady=15)
```

- Hàm hỗ trợ tạo frame có thanh cuộn dọc

```

def create_scrollable_frame(self, parent):
    """Hàm hỗ trợ tạo frame có thanh cuộn dọc (Scrollbar)"""
    canvas = tk.Canvas(parent)
    scrollbar = tk.Scrollbar(parent, orient='vertical', command=canvas.yview)
    inner_frame = tk.Frame(canvas)

    inner_frame.bind(
        '<Configure>',
        lambda e: canvas.configure(scrollregion=canvas.bbox('all'))
    )
    canvas.create_window((0, 0), window=inner_frame, anchor='nw')
    canvas.configure(yscrollcommand=scrollbar.set)

    scrollbar.pack(side=tk.RIGHT, fill=tk.Y)
    canvas.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)

    # Cập nhật cuộn bằng chuột
    def _on_mousewheel(event):
        # Chỉ cuộn canvas nếu con trỏ chuột đang nằm trên nó hoặc các con của nó
        try:
            widget = event.widget.winfo_containing(event.x_root, event.y_root)
            curr = widget
            while curr:
                if curr == canvas:
                    canvas.yview_scroll(int(-1*(event.delta/120)), 'units')
                    return
                curr = curr.master
            except Exception:
                pass

    canvas.bind_all('<MouseWheel>', _on_mousewheel, add='+')

    return canvas, scrollbar, inner_frame

```

- **tk.Canvas, tk.Scrollbar, tk.Frame**: Khởi tạo ba thành phần cốt lõi: vùng hiển thị (canvas), thanh cuộn dọc (scrollbar) và một khung chứa thực sự (inner_frame) nằm bên trong canvas.
- **inner_frame.bind('<Configure>', ...)**: Bắt sự kiện mỗi khi inner_frame thay đổi kích thước (ví dụ: khi chèn thêm hàng loạt các checkbox chỉ số). Lệnh này giúp tính toán lại giới hạn cuộn (scrollregion) để thanh cuộn bao quát hết dữ liệu mới.
- **canvas.create_window()**: Lệnh nhúng inner_frame vào bên trong không gian của canvas.
- **canvas.configure(yscrollcommand=...)**: Liên kết hoạt động của canvas với thanh cuộn để chúng đồng bộ với nhau.
- **Hàm _on_mousewheel và bind_all(...)**: Bắt sự kiện lăn chuột của người dùng. Thuật toán đi kèm sẽ rà soát vị trí của con trỏ, đảm bảo giao diện chỉ thực hiện lệnh cuộn lên/xuống khi chuột đang trỏ chính xác vào khu vực hiển thị danh sách.

- Hàm tìm kiếm dữ liệu cầu thủ

```

def search_player(self, player_num):
    entry = self.entry_p1 if player_num == 1 else self.entry_p2
    name = entry.get().strip()
    if not name:
        messagebox.showwarning('Error', 'Please enter a player name!')
        return

    try:
        # Gửi request lên API
        response = requests.get(f"{self.api_url}?name={name}")
        if response.status_code == 200:
            data = response.json().get('data', [])
            if data:
                player_data = data[0] # Lấy kết quả đầu tiên tìm được
                if player_num == 1:
                    self.data_p1 = player_data
                    self.display_stats(self.inner_p1, player_data)
                else:
                    self.data_p2 = player_data
                    self.display_stats(self.inner_p2, player_data)
            else:
                messagebox.showinfo('Info', 'Player not found!')
        else:
            messagebox.showerror('Error', f"Server error: {response.json().get('error', 'Unknown Error')}")
    except requests.exceptions.ConnectionError:
        messagebox.showerror('Error', 'Cannot connect to API. Make sure api.py is running!')
    except Exception as e:
        messagebox.showerror('Error', f"An error occurred: {e}")

```

- **entry.get().strip()**: Lấy chuỗi văn bản (tên cầu thủ) người dùng đã nhập vào ô **tk.Entry** và cắt bỏ các khoảng trắng thừa ở hai đầu. Nếu bỏ trống, hàm sẽ dùng messagebox để nhắc nhở và kết thúc sớm (return).
- **requests.get()**: Thực hiện gửi một HTTP GET request tới địa chỉ API (đã thiết lập ở phần khởi tạo), đính kèm tham số name để tìm kiếm cầu thủ tương ứng.
- **response.status_code == 200** và **response.json()**: Kiểm tra xem máy chủ API có trả về kết quả thành công (200) hay không. Nếu có, tiến hành giải mã dữ liệu trả về từ định dạng JSON sang cấu trúc dữ liệu của Python (Dictionary/List).
- Lưu trữ và gọi hàm hiển thị: Dữ liệu cầu thủ tìm được (kết quả đầu tiên **data[0]**) sẽ được lưu vào biến **self.data_p1** hoặc **self.data_p2** tùy thuộc vào luồng đang gọi. Sau đó, truyền dữ liệu này vào hàm **self.display_stats** (sẽ phân tích ở phần sau) để vẽ các hộp kiểm (checkbox) chỉ số lên màn hình.

- Hàm chuyển đổi sang float


```
def safe_float(self, val):
    """Hỗ trợ chuyển đổi an toàn sang float"""
    try:
        return float(str(val).replace(',', ''))
    except ValueError:
        return 0.0
```

- Hàm hiển thị dữ liệu cầu thủ

```
def display_stats(self, parent, player_data):
    # Xóa các widget cũ trong frame kết quả
    for widget in parent.winfo_children():
        widget.destroy()

    # Hiển thị thông tin cơ bản
    tk.Label(parent, text=f"Name: {player_data.get('Player', 'N/A')}", font=('Arial', 11, 'bold')).pack(anchor='w', padx=10, pady=(5,0))
    tk.Label(parent, text=f"Club: {player_data.get('Squad', 'N/A')}").pack(anchor='w', padx=10)
    tk.Label(parent, text='--- Stats ---', fg='blue').pack(anchor='w', padx=10, pady=(5, 5))

    # Bỏ qua những cột không phải là số/chỉ số (tùy vào dữ liệu thực tế)
    exclude_cols = ['Player', 'Nation', 'Pos', 'Squad', 'Age', 'Born', 'Matches']
```

- Xóa giao diện cũ (**widget.destroy()**): Vòng lặp duyệt qua tất cả các thành phần con trong khung chứa (parent) và xóa bỏ chúng. Bước này để làm sạch không gian trước khi hiển thị kết quả tìm kiếm mới, ngăn chặn tình trạng dữ liệu của cầu thủ cũ và mới bị chồng chéo lên nhau.
- Hiển thị thông tin cơ bản (**tk.Label**): Trích xuất và in ra các thông tin định danh như Tên (Player) và Câu lạc bộ (Squad) để người dùng xác nhận đã tìm đúng người.
- Lọc trường dữ liệu (**exclude_cols**): Khai báo một danh sách các cột mang tính chất thông tin cá nhân (như Quốc tịch, Vị trí, Tuổi...) thay vì điểm số thống kê. Các cột này sẽ bị bỏ qua trong quá trình tạo checkbox.

```

# Hiển thị tickbox cho mỗi chỉ số
for stat_name, stat_val in player_data.items():
    if stat_name in exclude_cols:
        continue

    try:
        # Đảm bảo là số hợp lệ thì mới hiện checkbox
        num_val = float(str(stat_val).replace(',', ''))

        frame = tk.Frame(parent)
        frame.pack(fill=tk.X, anchor='w', padx=15)

        # Tạo BooleanVar cho chỉ số nếu chưa có
        if stat_name not in self.check_vars:
            self.check_vars[stat_name] = tk.BooleanVar(value=False)

        cb = tk.Checkbutton(frame, text=f"{stat_name}: {num_val:g}", variable=self.check_vars[stat_name])
        cb.pack(side=tk.LEFT)
    except ValueError:
        pass

```

- Vòng lặp **for** để duyệt qua toàn bộ dữ liệu trong **player_data** và bỏ qua các trường không cần thiết thông qua **exclude_cols**.
- **try...except**: sử dụng để kiểm tra và chuyển dữ liệu sang kiểu số thực (**float**) nhằm đảm bảo chỉ những chỉ số hợp lệ mới được hiển thị. Nếu dữ liệu không hợp lệ, chương trình sẽ tự động bỏ qua mà không gây lỗi.
- Mỗi chỉ số sẽ được đặt trong một **Frame** riêng để giao diện hiển thị gọn gàng hơn. Sau đó, chương trình tạo biến trạng thái **BooleanVar** để lưu trạng thái checkbox của từng chỉ số.
- **tk.Checkbutton**: sử dụng để hiển thị checkbox kèm tên chỉ số và giá trị tương ứng lên giao diện.

- Hàm vẽ biểu đồ biểu đồ radar so sánh 2 cầu thủ

```

def compare(self):
    if not self.data_p1 or not self.data_p2:
        messagebox.showwarning('Warning', 'Need to search for 2 players to compare!')
        return

    # Lấy danh sách các chỉ số được chọn
    selected_stats = [stat for stat, var in self.check_vars.items() if var.get()]

    if len(selected_stats) < 3:
        messagebox.showwarning('Warning', 'Please select at least 3 stats to draw radar chart!')
        return

    name1 = self.data_p1.get('Player', 'Player 1')
    name2 = self.data_p2.get('Player', 'Player 2')

```

- Kiểm tra dữ liệu xem dữ liệu của cả hai cầu thủ đã được tải thành công hay chưa thông qua **self.data_p1** và **self.data_p2**. Nếu thiếu dữ liệu của một trong hai cầu thủ, hệ thống sẽ hiển thị thông báo cảnh báo và dừng quá trình so sánh.
- Duyệt qua từ điển trạng thái **self.check_vars** để lấy danh sách các chỉ số mà người dùng đã tích chọn. Các chỉ số này sẽ được lưu trong **selected_stats** để phục vụ cho quá trình so sánh và vẽ biểu đồ.
- Hệ thống yêu cầu người dùng phải chọn ít nhất 3 chỉ số. Đây là điều kiện tối thiểu để tạo thành một đa giác khép kín trên biểu đồ radar.
- Lấy thông tin hai cầu thủ từ dữ liệu đã lưu nhằm sử dụng làm nhãn chú thích (**label**) cho các đường biểu diễn trên biểu đồ radar.

```

values1 = []
values2 = []

# Lấy giá trị của các chỉ số đã chọn
for stat in selected_stats:
    v1 = self.safe_float(self.data_p1.get(stat, 0))
    v2 = self.safe_float(self.data_p2.get(stat, 0))
    values1.append(v1)
    values2.append(v2)

# Chuẩn hoá dữ liệu (thang 0 -> 1) để vẽ cùng trên biểu đồ radar
max_vals = [max(v1, v2, 1e-9) for v1, v2 in zip(values1, values2)]
norm_v1 = [v / m for v, m in zip(values1, max_vals)]
norm_v2 = [v / m for v, m in zip(values2, max_vals)]

# Lắp lại giá trị đầu tiên để đóng kín radar chart
norm_v1 += norm_v1[:1]
norm_v2 += norm_v2[:1]
angles = np.linspace(0, 2 * np.pi, len(selected_stats), endpoint=False).tolist()
angles += angles[:1]

```

- Duyệt qua danh sách **selected_stats** để lấy giá trị của từng chỉ số từ dữ liệu của hai cầu thủ bằng **self.data_p1.get()** và **self.data_p2.get()**. Các giá trị này được chuyển sang dạng số thực thông qua hàm **safe_float()** rồi lưu vào hai danh sách **values1** và **values2**.
- Thực hiện chuẩn hóa dữ liệu về cùng thang đo từ **0** → **1** bằng cách lấy giá trị lớn nhất của từng cặp chỉ số trong **max_vals** rồi chia từng giá trị cho giá trị lớn nhất tương ứng.
- Chương trình lắp lại giá trị đầu tiên vào cuối danh sách **norm_v1** và **norm_v2** nhằm tạo thành đa giác khép kín trên biểu đồ radar. Mảng

angles được tạo bằng **numpy.linspace()** để xác định góc hiển thị cho từng trục chỉ số trên biểu đồ.

```
# Vẽ biểu đồ radar với matplotlib
fig, ax = plt.subplots(figsize=(7, 7), subplot_kw=dict(polar=True))
ax.set_theta_offset(np.pi / 2) # Xoay góc bắt đầu lên đỉnh
ax.set_theta_direction(-1)     # Vẽ theo chiều kim đồng hồ

# Cài đặt nhãn cho các trục
ax.set_xticks(angles[:-1])
ax.set_xticklabels(selected_stats, fontsize=9)

# Vẽ Player 1
ax.plot(angles, norm_v1, label=name1, linewidth=2, linestyle='solid', color='blue')
ax.fill(angles, norm_v1, alpha=0.25, color='blue')

# Vẽ Player 2
ax.plot(angles, norm_v2, label=name2, linewidth=2, linestyle='solid', color='red')
ax.fill(angles, norm_v2, alpha=0.25, color='red')

ax.set_yticks([0.2, 0.4, 0.6, 0.8, 1.0])
ax.set_yticklabels(['20%', '40%', '60%', '80%', '100%'], color='grey', size=8)
ax.set_ylim(0, 1.0)

# Thêm ghi chú
ax.legend(loc='upper right', bbox_to_anchor=(1.3, 1.1))
plt.title(f"Comparison: {name1} vs {name2}", y=1.08, fontweight='bold')

plt.tight_layout()
plt.show()
```

- Sử dụng **matplotlib** để xây dựng biểu đồ radar thông qua hệ tọa độ cực (**polar**). Hàm **plt.subplots()** được sử dụng để tạo **Figure** và **Axes** với kích thước 7x7.

fig, ax = plt.subplots(figsize=(7, 7), subplot_kw=dict(polar=True))

- Thiết lập góc bắt đầu ở phía trên bằng **ax.set_theta_offset(np.pi / 2)** và vẽ theo chiều kim đồng hồ thông qua **ax.set_theta_direction(-1)** nhằm giúp biểu đồ hiển thị trực quan hơn.
- **ax.set_xticks()** và **ax.set_xticklabels()**: tạo nhãn cho các trục, trong đó mỗi trục biểu diễn một chỉ số thống kê đã được người dùng lựa chọn.
- **ax.plot()** để vẽ đường biểu diễn cho từng cầu thủ và **ax.fill()** để tô màu vùng bên trong đa giác nhằm giúp việc so sánh trực quan hơn. **Player 1** được hiển thị bằng màu xanh, **Player 2** được hiển thị bằng màu đỏ.

- Thiết lập các mốc giá trị trên trục bán kính bằng **ax.set_yticks()** và **ax.set_yticklabels()** với thang đo từ 20% đến 100%. Hàm **ax.set_ylim(0, 1.0)** được sử dụng để giới hạn dữ liệu trong khoảng từ 0 đến 1 sau khi chuẩn hóa.
- Cuối cùng, hệ thống sử dụng **ax.legend()** để hiển thị chú thích tên cầu thủ và **plt.title()** để tạo tiêu đề cho biểu đồ. Hàm **plt.tight_layout()** giúp căn chỉnh giao diện biểu đồ hợp lý trước khi hiển thị bằng **plt.show()**.

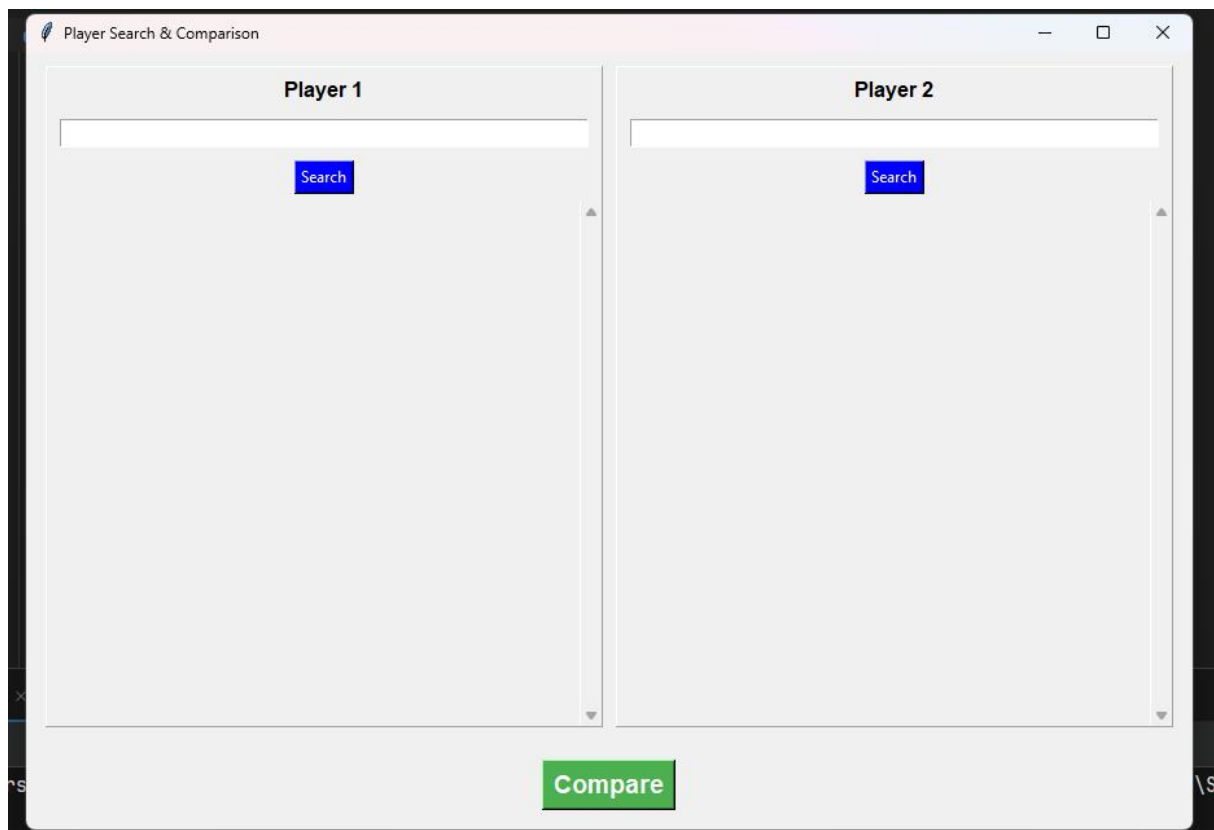
- Chạy chương trình

```
if __name__ == '__main__':
    root = tk.Tk()
    app = PlayerCompareGUI(root)
    root.mainloop()
```

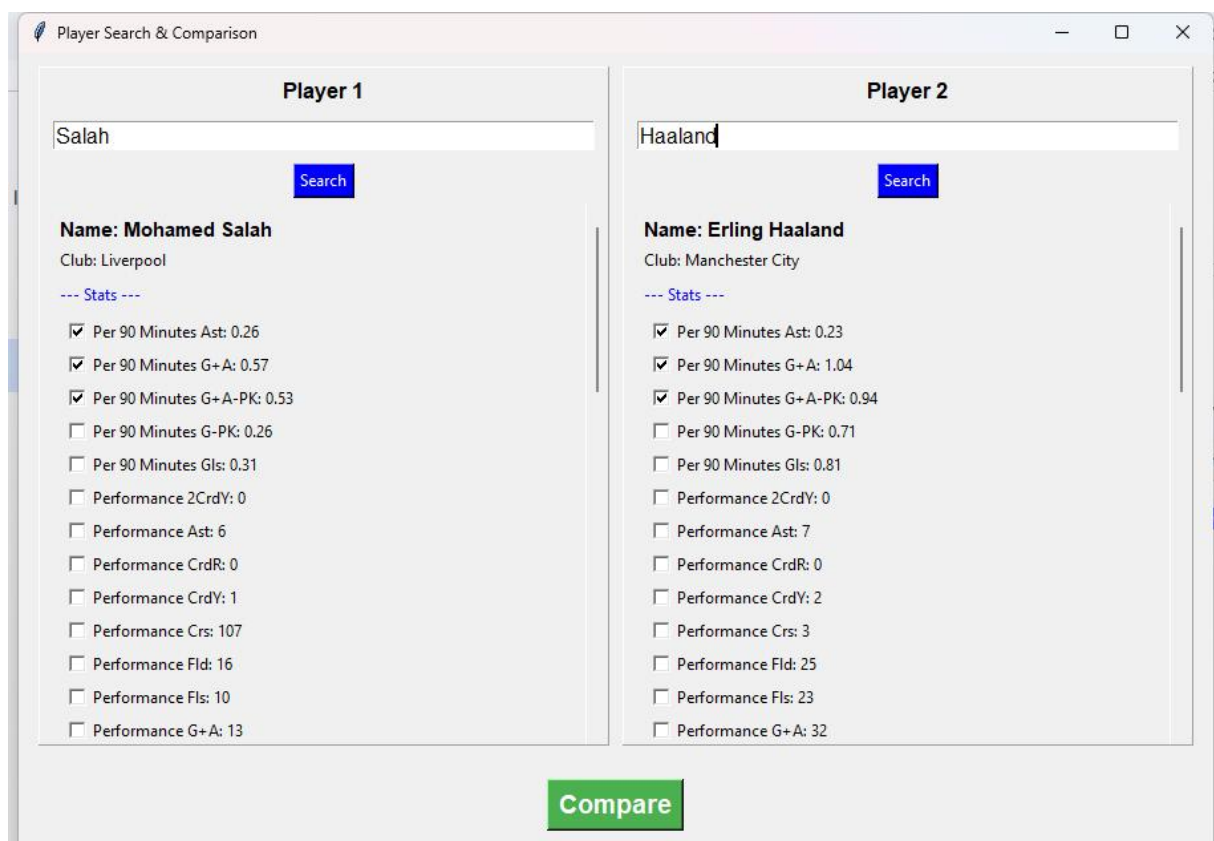
- Khởi tạo cửa sổ chính của **Tkinter** bằng **tk.Tk()**, sau đó tạo đối tượng **PlayerCompareGUI** để xây dựng toàn bộ giao diện và chức năng của ứng dụng.
- Cuối cùng, **root.mainloop()** được sử dụng để chạy vòng lặp chính của **Tkinter** nhằm duy trì giao diện hoạt động và chờ các thao tác từ người dùng như nhập dữ liệu, nhấn nút hoặc tương tác với checkbox.

- Một vài ảnh kết quả của giao diện

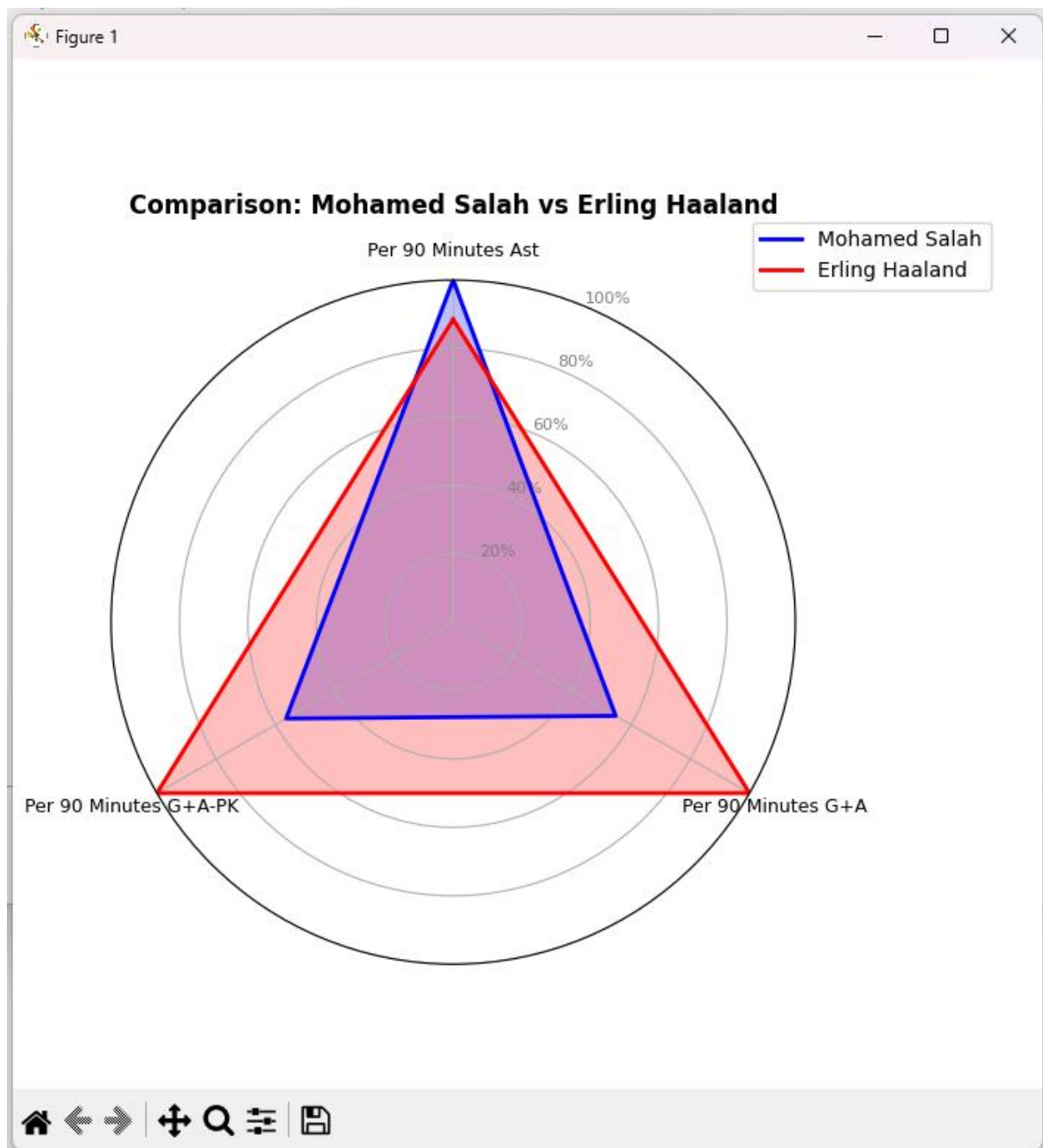
- Giao diện chính



- **Tìm kiếm cầu thủ và chọn chỉ số so sánh**



- **Vẽ biểu đồ radar so sánh**



CÂU 4: Bài tập này được bọn em lưu code vào file **analysis.py**

- Thư viện chung cần dùng

```
import os
import pandas as pd
```

- **os:** Cung cấp các hàm làm việc với hệ thống tệp, đặc biệt dùng để xử lý đường dẫn file trong chương trình.
- **pandas:** Thư viện lõi dùng để tạo, xử lý và phân tích dữ liệu dưới dạng bảng (DataFrame), hỗ trợ đọc/ghi file CSV và thực hiện các phép tính thống kê.

- Khởi tạo chung các đường dẫn truy cập đến thư mục và file

```
BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
OUTPUT_PATH = os.path.join(BASE_DIR, 'output')
DATA_PATH = os.path.join(OUTPUT_PATH, 'data.csv')
```

4.1. Tìm trung vị, trung bình và độ lệch chuẩn của mỗi chỉ số các cầu thủ của mỗi đội. Ghi kết quả vào 1 file CSV

4.1.1 Ý tưởng chung

Để làm bài tập này bọn em đã thực hiện việc phân tích dữ liệu cầu thủ bằng cách lựa chọn các chỉ số dạng số, sau đó nhóm dữ liệu theo từng đội bóng. Với mỗi đội, hệ thống tính toán các giá trị thống kê quan trọng gồm trung vị (median), trung bình (mean) và độ lệch chuẩn (standard deviation) cho từng chỉ số nhằm đánh giá mức độ điển hình và sự ổn định của đội. Các kết quả này được tổng hợp lại thành một bảng dữ liệu duy nhất, được sắp xếp khoa học theo từng chỉ số và cuối cùng được ghi ra file CSV để phục vụ cho việc phân tích và báo cáo.

4.1.2 Triển khai

- Đọc dữ liệu từ file CSV

```
df = pd.read_csv(DATA_PATH)
```

- Loại bỏ các cột chứa chữ (định danh), lấy tất cả các cột còn lại làm chỉ số tính toán

```
# Loại bỏ các cột chứa chữ (định danh), tự động lấy TẤT CẢ các cột còn lại làm chỉ số tính toán
INFO_COLS = ['Player', 'Nation', 'Pos', 'Squad', 'Age', 'Born']
NUMERIC_METRICS = [col for col in df.columns if col not in INFO_COLS]
```

- Ép kiểu số

```
for col in NUMERIC_METRICS:
    if col in df.columns:
        df[col] = pd.to_numeric(df[col], errors="coerce")
```

- Đoạn lệnh này duyệt qua danh sách các chỉ số số học (**NUMERIC_METRICS**) và kiểm tra xem từng cột có tồn tại trong **DataFrame** hay không. Nếu có, giá trị của cột đó sẽ được chuyển về dạng số bằng hàm **pd.to_numeric()**.
- Tham số **errors="coerce"** giúp chuyển các giá trị không hợp lệ (như "N/a") thành **NaN**.

- Tính median / mean / std theo từng đội

```
group = df.groupby('Squad')[NUMERIC_METRICS]

stats_median = group.median().add_suffix("_median")
stats_mean = group.mean().add_suffix("_mean")
stats_std = group.std().add_suffix("_std")
```

- Nhóm dữ liệu theo đội bóng: Sử dụng **groupby('Squad')** để gom các cầu thủ thuộc cùng một đội vào một nhóm, từ đó có thể thực hiện tính toán riêng cho từng đội.

- Tính các thống kê (median, mean, std): Sử dụng các hàm **median()**, **mean()** và **std()** để lần lượt tính trung vị, trung bình và độ lệch chuẩn cho mỗi chỉ số của từng đội.
- Chuẩn hóa tên cột: Sử dụng **add_suffix()** để thêm hậu tố **_median**, **_mean**, **_std** vào tên cột, giúp phân biệt rõ ràng giữa các loại thống kê trong bảng dữ liệu.

- Kết hợp các bảng thống kê

```
stats_all = pd.concat([stats_median, stats_mean, stats_std], axis=1).round(4)
```

- Ghép các bảng dữ liệu: Sử dụng **pd.concat()** để kết hợp ba DataFrame (**stats_median**, **stats_mean**, **stats_std**) thành một bảng duy nhất theo chiều ngang (**axis=1**).
- Làm tròn dữ liệu: Sử dụng **.round(4)** để làm tròn tất cả các giá trị đến 4 chữ số thập phân, giúp dữ liệu hiển thị gọn gàng và dễ đọc hơn.

- Sắp xếp lại thứ tự các cột

```
ordered_cols = []
for x in NUMERIC_METRICS:
    ordered_cols += [f"{x}_median", f"{x}_mean", f"{x}_std"]
stats_all = stats_all[ordered_cols]
```

- Tạo danh sách thứ tự cột: Duyệt qua từng chỉ số trong **NUMERIC_METRICS** và thêm vào danh sách **ordered_cols** theo thứ tự gồm ba cột: **_median**, **_mean**, **_std**. Ví dụ:

Playing Time	MP_median	MP_mean	MP_std

- Sắp xếp lại DataFrame: Sử dụng danh sách **ordered_cols** để sắp xếp lại các cột trong **stats_all**, đảm bảo các thống kê của cùng một chỉ số được đặt cạnh nhau.

- Ghi ra file CSV

```
csv_path = os.path.join(OUTPUT_PATH, 'team_stats.csv')
stats_all.fillna('N/A').to_csv(csv_path, encoding='utf-8')
print(f"Statistical results have been successfully saved!")
```

- Kết quả 1 phần của file team_stats.csv

	C1 ▾	±	C2 ▾	±	C3 ▾	±	C4 ▾	±	C5 ▾	±	C6 ▾	±	C7 ▾	±	t
	Squad		Playing Time MP_median		Playing Time MP_mean		Playing Time MP_std		Playing Time Starts_median		Playing Time Starts_mean		Playing Time Starts_std		f
2	Arsenal		25.5		23.2273		8.4512		18.0		17.0		10.5515		4
3	Aston Villa		21.0		20.8		8.9675		14.0		14.96		10.6945		5
4	Bournemouth		26.5		24.5		8.023		16.0		17.65		10.629		6
5	Brentford		29.0		23.3333		11.2086		21.0		17.7619		11.64		7
6	Brighton		23.5		21.7917		9.4776		16.5		15.5833		10.4129		8
7	Burnley		24.0		22.9545		7.6125		15.5		16.7273		8.7133		9
8	Chelsea		22.0		20.6		10.0125		12.0		14.92		10.9084		10
9	Crystal Palace		23.0		20.9048		9.6587		18.0		16.2381		11.4931		11
10	Everton		26.0		23.75		9.2502		19.0		18.6		11.3063		12
11	Fulham		26.0		24.4286		7.0255		17.0		17.619		9.917		13
12	Leeds United		24.5		23.4545		7.3919		17.0		17.0		10.4106		14
13	Liverpool		25.0		22.9545		9.4993		17.5		17.0		11.5511		15
14	Manchester City		20.0		19.7917		9.3296		15.0		14.6667		9.4899		16
15	Manchester Utd		23.0		22.7727		8.5353		14.5		17.0		9.9523		17
16	Newcastle United		24.5		23.7273		5.7832		17.0		16.9545		6.8832		18
17	Nottingham Forest		18.0		18.3333		11.0836		11.0		13.6667		11.8711		19
18	Sunderland		21.0		19.64		10.4199		14.0		14.92		10.6807		20
19	Tottenham Hotspur		22.5		19.9615		8.7108		13.0		14.3846		8.319		21
20	West Ham United		19.0		17.0		9.6648		12.5		13.3571		9.5072		22
21	Wolves		21.0		18.8519		8.7561		14.0		13.8519		7.7196		23

4.2 Tìm đội bóng có chỉ số điểm số cao nhất ở mỗi chỉ số. Theo bạn đội nào có phong độ tốt nhất giải ngoại Hạng Anh mùa 2025-2026

4.2.1 Ý tưởng chung

Trước tiên để tìm đội bóng có chỉ số cao nhất ở mỗi chỉ số, bọn em thực hiện bằng cách sử dụng bảng giá trị trung bình của từng đội theo từng chỉ số, sau đó loại bỏ các cột không hợp lệ (toàn giá trị thiếu). Với mỗi chỉ số, chương trình xác định đội có giá trị lớn nhất bằng cách so sánh giữa các đội, đồng thời ghi nhận cả giá trị tương ứng. Kết quả được tổng hợp thành một bảng, trong đó mỗi dòng thể hiện một chỉ số, đội bóng dẫn đầu và giá trị cao nhất.

Tiếp theo đến với việc xây dựng hệ thống đánh giá phong độ đội bóng, bọn em thực hiện bằng cách tổng hợp nhiều chỉ số thống kê quan trọng liên quan đến tấn công, phòng ngự, kỷ luật và hiệu quả thi đấu. Các chỉ số được chia thành nhóm tích cực và tiêu cực, sau đó được chuẩn hóa về cùng một thang điểm để đảm bảo tính công bằng khi so sánh. Mỗi chỉ số được gán trọng số khác nhau tùy theo mức độ quan trọng trong thực tế bóng đá. Cuối cùng, hệ thống

tính tổng điểm phong độ cho từng đội bóng và tạo bảng xếp hạng từ cao xuống thấp.

4.2.2 Triển khai

- Tìm đội bóng có chỉ số điểm số cao nhất ở mỗi chỉ số

- Tính giá trị trung bình theo từng đội (lấy sau dấu phẩy 4 chữ số)

```
mean_df = group.mean().round(4)
```

- Loại bỏ những cột mà tất cả giá trị là NaN

```
mean_df_clean = mean_df.dropna(axis=1, how='all')
```

- Xác định đội bóng và giá trị cao nhất theo từng chỉ số

```
# Hàm idxmax trả về tên đội bóng có chỉ số lớn nhất trong cột đó
best_team_per_metric = mean_df_clean.idxmax(skipna=True)

#Hàm max trả về chỉ số
best_val_per_metric = mean_df_clean.max(skipna=True)
```

- Xác định đội dẫn đầu: Sử dụng **idxmax()** để tìm tên đội bóng có giá trị lớn nhất ở mỗi cột (mỗi chỉ số).
- Lấy giá trị lớn nhất: Sử dụng **max()** để lấy giá trị cao nhất tương ứng của từng chỉ số.
- Xử lý dữ liệu thiếu: Tham số **skipna=True** giúp bỏ qua các giá trị NaN trong quá trình tính toán.
- Tạo bảng kết quả đội dẫn đầu theo từng chỉ số

```
best_table = pd.DataFrame({
    "Metric"          : best_team_per_metric.index,
    "Best Team"       : best_team_per_metric.values,
    "Mean value"      : best_val_per_metric.values.round(4),
})
```

- Khởi tạo DataFrame: Sử dụng **pd.DataFrame()** để tạo bảng dữ liệu tổng hợp từ các kết quả đã tính toán.
- Cột "**Metric**": Lưu tên các chỉ số (tên cột) lấy từ index của **best_team_per_metric**.
- Cột "**Best Team**": Lưu tên đội bóng có giá trị cao nhất tương ứng với từng chỉ số.
- Cột "**Mean value**": Lưu giá trị trung bình cao nhất của mỗi chỉ số, được làm tròn đến 4 chữ số thập phân.

- Ghi bảng kết quả đội dẫn đầu theo từng chỉ số ra file CSV

```
best_csv = os.path.join(OUTPUT_PATH, 'best_team_per_stat.csv')
best_table.to_csv(best_csv, index=False, encoding="utf-8-sig")
print(f"Best team per stat has been successfully saved!")
```

- Kết quả 1 phần của file best_team_per_stat.csv

	Metric ∇	÷	Best Team ∇	÷	Mean value ∇	÷
1	Playing Time MP		Bournemouth		24.5	
2	Playing Time Starts		Everton		18.6	
3	Playing Time Min		Burnley		728.6667	
4	Playing Time 90s		Everton		18.555	
5	Performance GlS		Arsenal		2.6818	
6	Performance Ast		Manchester City		2.0833	
7	Performance G+A		Arsenal		4.6818	
8	Performance G-PK		Arsenal		2.5	
9	Performance PK		Brentford		0.3333	
10	Performance PKatt		Brentford		0.4286	
11	Performance CrdY		Bournemouth		3.7	
12	Performance CrdR		Chelsea		0.28	
13	Per 90 Minutes GlS		Brighton		0.1646	
14	Per 90 Minutes Ast		Brentford		0.1486	
15	Per 90 Minutes G+A		Arsenal		0.2886	
16	Per 90 Minutes G-PK		Liverpool		0.16	
17	Per 90 Minutes G+A-PK		Liverpool		0.2864	
18	Performance GA		Burnley		68.0	
19	Performance GA90		Wolves		2.2233	
20	Performance SoTA		Burnley		184.0	
21	Performance Saves		Burnley		122.0	
22	Performance Save%		Crystal Palace		85.9	
23	Performance W		Arsenal		22.0	
24	Performance D		Bournemouth		16.0	
25	Performance L		Burnley		22.0	
26	Performance CS		Arsenal		16.0	
27	Performance CS%		Crystal Palace		67.2	
28	Penalty Kicks PKatt		Burnley		7.0	

- Tìm đội bóng có phong độ tốt nhất giải ngoại hạng Anh

- Phân loại chỉ số đánh giá phong độ

```

FORM_POSITIVE = [
    'Performance GLs', 'Performance Ast', 'Standard SoT', 'Standard SoT%', 'Standard G/Sh',
    'Team Success PPM', 'Team Success +/-', 'Performance TklW', 'Performance Int', 'Performance Fld',
]

FORM_NEGATIVE = [
    'Performance CrdY', 'Performance CrdR', 'Performance Fls',
    'Performance OG', 'Team Success onGA',
]

```

- **FORM_POSITIVE**: chứa các chỉ số tích cực, tức là giá trị càng cao thì phong độ đội bóng càng tốt. Nhóm này bao gồm các chỉ số tấn công, phòng ngự và hiệu quả thi đấu như bàn thắng, kiến tạo, số cú sút trúng đích, điểm trung bình mỗi trận và số pha tắc bóng thành công.
- **FORM_NEGATIVE**: chứa các chỉ số tiêu cực, tức là giá trị càng thấp thì đội bóng thi đấu càng hiệu quả. Nhóm này gồm các chỉ số như thẻ vàng, thẻ đỏ, phạm lỗi, phản lưới nhà và số bàn thua của đội.

● Thiết lập trọng số đánh giá

```

# Tổng trọng số của tất cả các chỉ số nên bằng 1.0 (hoặc 100%)
WEIGHTS = {
    # Hiệu suất chung (Quan trọng nhất)
    'Team Success PPM': 0.30,
    'Team Success +/-': 0.15,
    # Tấn công
    'Performance GLs': 0.12,
    'Performance Ast': 0.08,
    'Standard SoT': 0.05,
    'Standard SoT%': 0.04,
    'Standard G/Sh': 0.04,
    'Performance Fld': 0.02,
    # Phòng ngự & Kỷ luật
    'Team Success onGA': 0.10,
    'Performance TklW': 0.04,
    'Performance Int': 0.04,
    'Performance CrdR': 0.01,
    'Performance CrdY': 0.005,
    'Performance Fls': 0.005,
    'Performance OG': 0.01,
}

```

- **Hiệu suất chung (45%):** Là nhóm chỉ số quan trọng nhất nên chiếm tổng trọng số 45%. Trong đó, **Team Success PPM** được gán 30% do phản ánh trực tiếp số điểm trung bình mà đội bóng giành được mỗi trận, còn **Team Success +/-** chiếm 15% nhằm đánh giá hiệu quả chênh lệch bàn thắng và bàn thua của đội.
- **Nhóm chỉ số tấn công (35%):** Các chỉ số như **Performance GlS**, **Performance Ast**, **Standard SoT**, **Standard SoT%** và **Standard G/Sh** được sử dụng để đánh giá khả năng ghi bàn, tạo cơ hội và hiệu suất dứt điểm của đội bóng trong suốt mùa giải.
- **Nhóm phòng ngự và kỷ luật (20%):** Các chỉ số như **Performance TklW**, **Performance Int** và **Team Success onGA** phản ánh khả năng phòng ngự và thu hồi bóng của đội. Ngoài ra, các thông số tiêu cực như thẻ đỏ, thẻ vàng, phạm lỗi và phản lưới nhà cũng được đưa vào nhằm đánh giá tính kỷ luật trong lối chơi.

- **Gom nhóm và chuẩn hóa dữ liệu đội bóng về thang (0 - 1)**

```
grouped = df.groupby('Squad')[FORM_POSITIVE + FORM_NEGATIVE].mean()

normalized = pd.DataFrame(index=grouped.index)
```

- **Chuẩn hóa Min-Max theo hướng dữ liệu**

```
# Nhóm càng cao càng tốt
for col in FORM_POSITIVE:
    min_val, max_val = grouped[col].min(), grouped[col].max()
    if max_val != min_val:
        normalized[col] = (grouped[col] - min_val) / (max_val - min_val)
    else:
        normalized[col] = 1.0

# Nhóm càng thấp càng tốt (Tự động đảo chiều điểm số không cần nhân -1 trước)
for col in FORM_NEGATIVE:
    min_val, max_val = grouped[col].min(), grouped[col].max()
    if max_val != min_val:
        normalized[col] = (max_val - grouped[col]) / (max_val - min_val)
    else:
        normalized[col] = 1.0
```

- Với nhóm chỉ số **FORM_POSITIVE**, chương trình sử dụng công thức Min-Max để chuẩn hóa dữ liệu về thang điểm từ 0 → 1. Các giá trị càng cao sẽ nhận điểm càng lớn nhằm phản ánh những yếu tố có lợi như ghi bàn, kiến tạo hoặc khả năng phòng ngự.
- Với nhóm chỉ số **FORM_NEGATIVE**, công thức chuẩn hóa được đảo chiều để các đội bóng có ít thẻ phạt, ít phạm lỗi hoặc ít bàn thua hơn sẽ nhận điểm cao hơn. Điều kiện **if max_val != min_val** cũng được sử dụng để tránh lỗi chia cho 0 khi dữ liệu của một chỉ số không có sự khác biệt.

● Tính điểm phong độ tổng hợp

```
for col in WEIGHTS.keys():
    normalized[col] = normalized[col] * WEIGHTS[col]

team_ranking = normalized[list(WEIGHTS.keys())].sum(axis=1) * 100
```

- Tiến hành nhân từng chỉ số đã chuẩn hóa với trọng số tương ứng trong **WEIGHTS** để phản ánh mức độ quan trọng khác nhau của từng yếu tố trong đánh giá phong độ. Sau đó, toàn bộ các điểm thành phần sẽ được cộng lại để tạo ra điểm phong độ tổng hợp cho mỗi đội bóng. Kết quả cuối cùng được nhân với 100 nhằm đưa hệ thống về thang điểm trực quan từ 0 → 100

● Sắp xếp thứ hạng và Xuất dữ liệu

```
# Xuất kết quả định dạng chuẩn
ranking = (
    team_ranking.sort_values(ascending=False)
    .round(2)
    .reset_index()
)
ranking.columns = ['Team', 'Score']
ranking.index += 1
ranking.index.name = 'Rank'

rank_csv = os.path.join(OUTPUT_PATH, 'team_form_ranking.csv')
ranking.to_csv(rank_csv, encoding='utf-8')
print(f"Ranking results have been successfully saved!")
```

- Sử dụng **sort_values(ascending=False)** để sắp xếp điểm phong độ của các đội bóng theo thứ tự giảm dần, sau đó làm tròn dữ liệu đến 2 chữ số thập phân bằng **round(2)**. **reset_index()** giúp tạo lại chỉ số và đổi tên các cột thành **Team** và **Score** để bảng xếp hạng rõ ràng hơn.
- Sau khi hoàn thiện bảng xếp hạng, hệ thống gán số thứ hạng (**Rank**) cho từng đội bóng và sử dụng **to_csv()** để xuất kết quả thành file **team_form_ranking.csv**.

● **Kết quả của file team_form_ranking.csv**

	Rank ∇	÷	Team ∇	÷	Score ∇	÷
1		1	Arsenal		88.84	
2		2	Manchester City		78.86	
3		3	Manchester Utd		72.36	
4		4	Liverpool		64.5	
5		5	Bournemouth		60.7	
6		6	Brentford		59.66	
7		7	Everton		57.73	
8		8	Brighton		55.92	
9		9	Aston Villa		54.26	
10		10	Chelsea		51.74	
11		11	Newcastle United		51.07	
12		12	Leeds United		48.33	
13		13	Fulham		45.74	
14		14	Tottenham Hotspur		44.54	
15		15	Crystal Palace		40.52	
16		16	Nottingham Forest		39.84	
17		17	Sunderland		38.49	
18		18	West Ham United		28.71	
19		19	Burnley		18.24	
20		20	Wolves		11.34	

=> KẾT LUẬN: Đội bóng có phong độ cao nhất mùa 2025-2026 là Arsenal

CÂU 5: Bài tập này được em lưu vào file **kmeans.py**

5.1 Ý tưởng chung

Để phân nhóm và khám phá vai trò chiến thuật của cầu thủ, chương trình bọn em sử dụng thuật toán K-means kết hợp với phương pháp giảm chiều dữ liệu PCA. Tuy nhiên, ở mô hình thử nghiệm ban đầu, dữ liệu bị nhiễu nặng do bộ chỉ số đặc thù của vị trí Thủ môn (tỷ lệ cứu thua rất cao nhưng các thông số tấn công, di chuyển đều bằng 0). Sự chênh lệch không gian đặc trưng này làm biến dạng thước đo khoảng cách của thuật toán, khiến K-means gom cụm sai lệch (điển hình như việc xếp thủ môn xuất sắc vào chung nhóm với hậu vệ dự bị).

Để khắc phục, bọn em áp dụng chiến lược "Chia để trị": cô lập vị trí thủ môn và tách bài toán thành hai không gian độc lập là Nhóm cầu thủ tuyến trên (Outfield Players) và Nhóm thủ môn (Goalkeepers). Nhờ vậy, trên từng tập dữ liệu, đồ thị Elbow và điểm Silhouette Score đã hoạt động hiệu quả để xác định chính xác số cụm (k) tối ưu. Sau khi K-means phân nhóm, phương pháp PCA được dùng để nén và trực quan hóa dữ liệu trên không gian 2D/3D.

5.2 Triển khai

- Khai báo thư viện

```
import os
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from yellowbrick.cluster import SilhouetteVisualizer
```

- **os**: Cung cấp các hàm làm việc với hệ thống tệp, đặc biệt dùng để xử lý đường dẫn file trong chương trình.
- **pandas**: Thư viện lõi dùng để tạo, xử lý và lưu trữ dữ liệu dưới dạng bảng (DataFrame).
- **matplotlib.pyplot** và **seaborn**: Thư viện trực quan hóa dữ liệu, dùng để vẽ đồ thị (Elbow, phân bố Silhouette, Scatter plot 2D/3D).

- **StandardScaler**: Xử lý chuẩn hóa dữ liệu, đưa các chỉ số cầu thủ về cùng hệ quy chiếu để thuật toán tính toán chính xác, không bị lệch do chênh lệch thang đo.
- **KMeans**: Thuật toán học máy không giám sát cốt lõi, làm nhiệm vụ phân nhóm các cầu thủ có đặc điểm chiến thuật tương đồng.
- **PCA**: Thuật toán giảm chiều dữ liệu, giúp nén không gian hàng chục chỉ số xuống 2D hoặc 3D để dễ dàng biểu diễn trực quan.
- **SilhouetteVisualizer**: Công cụ vẽ đồ thị đánh giá hiệu suất phân cụm, hỗ trợ đối chiếu và xác nhận số cụm k tối ưu.

- Đọc và phân tách dữ liệu

```
print("Reading data...")
df = pd.read_csv(DATA_PATH)

df_out = df[~df['Pos'].str.contains('GK', na=False)].copy().reset_index(drop=True)
df_gk = df[df['Pos'].str.contains('GK', na=False)].copy().reset_index(drop=True)
```

- **df_out**: Áp dụng toán tử phủ định (~) để lọc bỏ các dòng có vị trí 'Pos' là Thủ môn 'GK', qua đó tách riêng được tập dữ liệu của các Cầu thủ tuyến trên.
- **df_gk**: Trích xuất tập dữ liệu dành riêng cho Thủ môn (các dòng có chứa chữ 'GK').
- **.copy().reset_index(drop=True)**: Đảm bảo tạo ra một bản sao dữ liệu hoàn toàn độc lập và đặt lại thứ tự chỉ số (index) từ 0, giúp tránh các lỗi tham chiếu bộ nhớ khi chạy thuật toán phía sau.

- Phân loại chỉ số theo vị trí cầu thủ

```

OUTFIELD = [
    # --- CHỈ SỐ TẤN CÔNG (ATK) ---
    'Per 90 Minutes Gls', # Bàn thắng ghi được mỗi 90 phút
    'Per 90 Minutes Ast', # Số pha kiến tạo mỗi 90 phút
    'Standard Sh/90', # Số cú sút tung ra mỗi 90 phút
    'Standard SoT/90', # Số cú sút trúng đích mỗi 90 phút
    'Standard G/Sh', # Tỷ lệ bàn thắng trên mỗi cú sút (Hiệu suất dứt điểm)
    'Performance Off', # Số lần bị thổi việt vị
    'Performance Fld', # Số lần bị phạm lỗi (mang lại quả phạt cho đội)

    # --- CHỈ SỐ PHÒNG NGỰ (DEF) ---
    'Performance TklW', # Số pha tắc bóng (thủ hồi bóng) thành công
    'Performance Int', # Số lần cắt bóng / đánh chặn
    'Performance Fls', # Số lần phạm lỗi với đối phương
    'Performance CrdY', # Số thẻ vàng phải nhận
    'Performance CrdR', # Số thẻ đỏ phải nhận
    'Performance Crs', # Số lượng quả tạt bóng

    # --- CHỈ SỐ ĐÓNG GÓP TẬP THỂ (TEAM) ---
    'Team Success PPM', # Điểm số trung bình giành được mỗi trận khi cầu thủ ra sân (Points Per Match)
    'Team Success +/-90', # Hiệu số bàn thắng/thua của đội mỗi 90 phút khi cầu thủ có mặt trên sân
    # "Playing Time 90s", # Tổng số thời gian thi đấu (quy đổi ra số trận 90 phút)
]

GK = [
    # --- CHỈ SỐ THỦ MÔN (GOALKEEPER) ---
    'Performance Save%', # Tỷ lệ cứu thua thành công (%)
    'Performance GA90', # Số bàn thua trung bình mỗi 90 phút (Goals Against)
    'Performance CS%', # Tỷ lệ phần trăm số trận giữ sạch lưới (Clean Sheets)
    'Performance W', # Số trận thắng bất chính
    'Performance D', # Số trận hòa bất chính
    'Performance L', # Số trận thua bất chính
    'Penalty Kicks Save%', # Tỷ lệ cản phá phạt đền (Penalty) thành công (%)
    'Performance Saves', # Tổng số pha cứu thua thành công
]

```

- **OUTFIELD:** chứa các chỉ số dành cho cầu thủ thi đấu ngoài sân như tiền đạo, tiền vệ và hậu vệ.
- **GK:** chứa các chỉ số chuyên biệt dành cho thủ môn như tỷ lệ cứu thua, tỷ lệ giữ sạch lưới hoặc số bàn thua trung bình mỗi 90 phút.

- Hàm tiền xử lý dữ liệu

```

print('Preprocessing data...')
def preprocess_features(data, columns):
    numeric_data = data[columns].copy()
    # Làm sạch các dấu phẩy và ép kiểu về số
    for col in columns:
        numeric_data[col] = numeric_data[col].astype(str).str.replace(',', '')
        numeric_data[col] = pd.to_numeric(numeric_data[col], errors='coerce')
    # Loại bỏ các cột toàn NaN và điền 0 cho các giá trị NaN còn lại
    numeric_data = numeric_data.dropna(axis=1, how='all').fillna(0)
    scaler = StandardScaler()
    scaled_data = scaler.fit_transform(numeric_data)
    return numeric_data, scaled_data

```

- **replace, to_numeric**: Lọc ra các cột dữ liệu cần thiết, loại bỏ dấu phẩy trong chuỗi bằng **str.replace(',', '')** và chuyển dữ liệu sang dạng số bằng **pd.to_numeric()**. Tham số **errors='coerce'** giúp chuyển các giá trị lỗi hoặc không hợp lệ thành NaN.
 - Loại bỏ các cột chứa toàn giá trị rỗng (NaN) bằng **dropna(axis=1, how='all')**. Các giá trị thiếu còn lại được thay thế bằng 0 thông qua **fillna(0)**.
 - Sử dụng **StandardScaler()** để đưa các chỉ số về cùng một thang đo chuẩn (**Z-score**). Bước này giúp thuật toán **K-Means** đo khoảng cách chính xác hơn và tránh bị ảnh hưởng bởi các chỉ số có độ lớn quá chênh lệch.
 - Trả kết quả (return) về đồng thời hai bộ dữ liệu gồm:
 - **numeric_data**: dữ liệu số đã làm sạch
 - **scaled_data**: dữ liệu đã được chuẩn hóa để sử dụng cho quá trình huấn luyện mô hình **K-Means**.
- Gọi hàm **preprocess_features()** để làm sạch và chuẩn hóa dữ liệu cho hai nhóm cầu thủ gồm cầu thủ ngoài sân (OUTFIELD) và thủ môn (GK).

```
X, X_scaled = preprocess_features(df_out, OUTFIELD)
X_gk, X_gk_scaled = preprocess_features(df_gk, GK)
```

- Hàm vẽ biểu đồ Elbow

```
# Hàm vẽ Elbow plot
def elbow_plot(X_scaled_data, title, output_path, file_name, kmin=1, kmax=10):
    distortions = []
    K_range = range(kmin, kmax + 1)
    for k in K_range:
        kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
        kmeans.fit(X_scaled_data)
        distortions.append(kmeans.inertia_)

    plt.figure(figsize=(10, 6))
    plt.plot(K_range, distortions, 'bx-')
    plt.xlabel('Number of clusters (k)')
    plt.ylabel('Distortion')
    plt.title(title)
    plt.grid(True)
    out_path = os.path.join(output_path, file_name)
    plt.savefig(out_path)
    plt.show()
    plt.close()
    print('Elbow plot generated successfully.')
```

- **for k in K_range:** Khởi tạo và chạy thử nghiệm thuật toán K-means nhiều lần với số lượng cụm **k** tăng dần từ **kmin** đến **kmax**.
- **random_state=42** giúp kết quả ổn định sau mỗi lần chạy, **n_init=10** giúp K-Means chọn kết quả phân cụm tốt hơn.
- **kmeans.inertia_:** Ở mỗi vòng lặp, thuật toán đo lường tổng bình phương khoảng cách từ các điểm dữ liệu đến tâm cụm của chúng (Inertia/Distortion). Khoảng cách này càng nhỏ nghĩa là các phần tử trong cụm càng có sự tương đồng cao.
- **plt.plot:** Vẽ đồ thị đường biểu diễn sự thay đổi của độ phân tán theo số lượng cụm **k**.
- Cuối cùng, biểu đồ được lưu bằng **plt.savefig()** và hiển thị bằng **plt.show()**.

- Hàm vẽ biểu đồ Silhouette

```
# Hàm vẽ Silhouette plot
def silhouette_multi_plot(X_scaled_data, title, output_path, file_name, k_min=2, k_max=5):
    num_k = k_max - k_min + 1
    cols = 2
    rows = (num_k + 1) // cols

    fig, axes = plt.subplots(rows, cols, figsize=(15, 6 * rows))
    fig.suptitle(title, fontsize=16)

    for i, k in enumerate(range(k_min, k_max + 1)):
        ax = axes[i // cols, i % cols] if rows > 1 else axes[i % cols]
        model = KMeans(n_clusters=k, random_state=42, n_init=10)
        visualizer = SilhouetteVisualizer(model, ax=ax, colors='yellowbrick', force_model=True)
        visualizer.fit(X_scaled_data)
        visualizer.finalize()
        ax.set_title(f"k = {k}")

    fig.tight_layout()
    out_path = os.path.join(output_path, file_name)
    fig.savefig(out_path)
    plt.show()
    plt.close(fig)
    print('Silhouette plots created successfully.')
```

- **fig, axes = plt.subplots(rows, cols, figsize=(15, 6 * rows))**: Thiết lập không gian hiển thị động, tính toán số hàng và số cột dựa trên khoảng giá trị k để tạo nhiều biểu đồ con (subplot) trong cùng một cửa sổ hiển thị.
- **for i, k in enumerate(range(k_min, k_max + 1))**: Hiển thị nhiều biểu đồ **Silhouette** trên cùng một màn hình giúp dễ dàng so sánh chất lượng phân cụm giữa các giá trị k khác nhau.
- Với mỗi giá trị k, mô hình K-Means được khởi tạo và đưa vào **SilhouetteVisualizer** để tính toán hệ số **Silhouette**, giúp đánh giá mức độ kết dính và tách biệt giữa các cụm.
- **tight_layout()** được sử dụng để căn chỉnh bố cục, sau đó biểu đồ được hiển thị bằng **plt.show()** và lưu thành file ảnh bằng **fig.savefig()**.

- Sử dụng thuật toán PCA vẽ scatter plot phân cụm dữ liệu trên mặt 2D

```
# Sử dụng thuật toán PCA vẽ scatter plot phân cụm dữ liệu trên mặt 2D
def plot_pca_2d(df, X_scaled_data, title, output_path, file_name, cluster_col='Cluster'):
    pca = PCA(n_components=2)
    X_pca = pca.fit_transform(X_scaled_data)
    df['PCA1_2D'] = X_pca[:, 0]
    df['PCA2_2D'] = X_pca[:, 1]

    plt.figure(figsize=(10, 8))
    sns.scatterplot(x='PCA1_2D', y='PCA2_2D', hue=cluster_col, palette='viridis', data=df, s=80, alpha=0.8)
    plt.title(title)
    plt.xlabel('Principal Component 1 (PCA1)')
    plt.ylabel('Principal Component 2 (PCA2)')
    plt.legend(title='Cluster')
    plt.grid(True)
    out_path = os.path.join(output_path, file_name)
    plt.savefig(out_path)
    plt.show()
    plt.close()
    print('PCA 2D plot generated successfully.')
```

- **PCA(n_components=2)**: Thuật toán Phân tích thành phần chính (PCA) nén tập dữ liệu gốc (gồm rất nhiều chỉ số thống kê của cầu thủ) xuống chỉ còn 2 chiều cơ bản.
- Kết quả PCA được lưu vào DataFrame dưới dạng hai cột tọa độ mới.
 - `df['PCA1_2D'] = pca_result[:, 0]`
 - `df['PCA2_2D'] = pca_result[:, 1]`
- **sns.scatterplot**: Sử dụng thư viện Seaborn để vẽ đồ thị phân tán. Tham số **hue=cluster_col**: tự động phân loại và tô màu các điểm dựa trên nhãn cụm từ thuật toán **K-Means** trước đó. Điều này giúp người xem dễ dàng quan sát mức độ tập trung, sự tách biệt hoặc ranh giới giữa các nhóm cầu thủ khác nhau.
- Thêm các thuộc tính cho biểu đồ như tiêu đề, nhãn các trục thành phần chính (**PCA1**, **PCA2**), bảng chú thích màu (**legend**), sau đó tự động xuất kết quả thành file ảnh (**plt.savefig**).

- Sử dụng thuật toán PCA vẽ scatter plot phân cụm dữ liệu trên mặt 3D


```
# Sử dụng thuật toán PCA vẽ scatter plot phân cụm dữ liệu trên mặt 3D
def plot_pca_3d(df, X_scaled_data, title, output_path, file_name, cluster_col='Cluster'):
    pca = PCA(n_components=3)
    X_pca = pca.fit_transform(X_scaled_data)
    df['PCA1_3D'] = X_pca[:, 0]
    df['PCA2_3D'] = X_pca[:, 1]
    df['PCA3_3D'] = X_pca[:, 2]

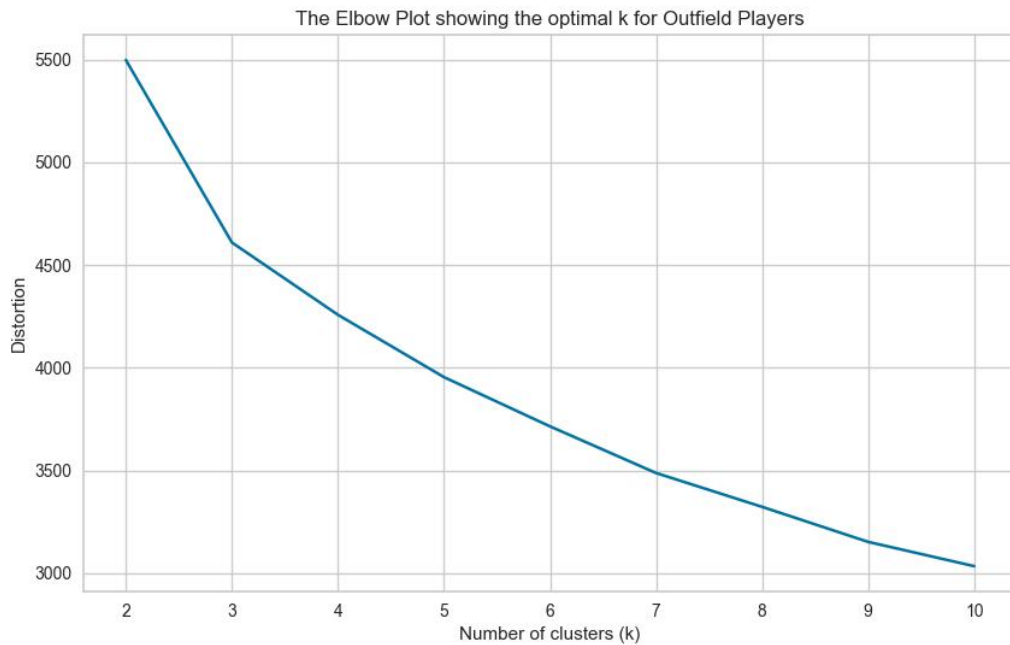
    fig = plt.figure(figsize=(10, 8))
    ax = fig.add_subplot(111, projection='3d')
    scatter = ax.scatter(df['PCA1_3D'], df['PCA2_3D'], df['PCA3_3D'], c=df[cluster_col], cmap='viridis', s=40, alpha=0.8)
    ax.set_title(title)
    ax.set_xlabel('PCA1')
    ax.set_ylabel('PCA2')
    ax.set_zlabel('PCA3')
    legend = ax.legend(*scatter.legend_elements(), title='Cluster')
    ax.add_artist(legend)
    out_path = os.path.join(output_path, file_name)
    plt.savefig(out_path)
    plt.show()
    plt.close()
    print('PCA 3D plot generated successfully.')
```

- **PCA(n_components=3)**: Thuật toán PCA nén tập dữ liệu các chỉ số thống kê phức tạp xuống còn 3 thành phần chính cốt lõi.
- Kết quả PCA được trích xuất và lưu vào 3 cột mới (**PCA1_3D**, **PCA2_3D**, **PCA3_3D**) trong DataFrame. Ba cột này đóng vai trò tương ứng là trục X, Y và Z để xác định tọa độ không gian của từng cá nhân.
- **projection='3d' & ax.scatter**: Thiết lập một hệ trục tọa độ 3 chiều thông qua Matplotlib. Đồ thị phân tán (scatter plot) sẽ biểu diễn mỗi cầu thủ là một điểm trong không gian. Màu sắc của các điểm (**cmap='viridis'**) được phân loại tự động dựa trên nhãn cụm (Cluster) từ K-Means, giúp người xem xoay và quan sát rõ cấu trúc, ranh giới của các nhóm từ nhiều góc độ.
- Định danh cho cả 3 trục (X, Y, Z), gắn bảng chú thích (legend) để giải nghĩa các nhóm màu, sau đó tự động lưu biểu đồ thành file ảnh (**plt.savefig**).

5.3 Kết quả

5.3.1 Phân cụm đối với cầu thủ tuyển trên (Outfield players)

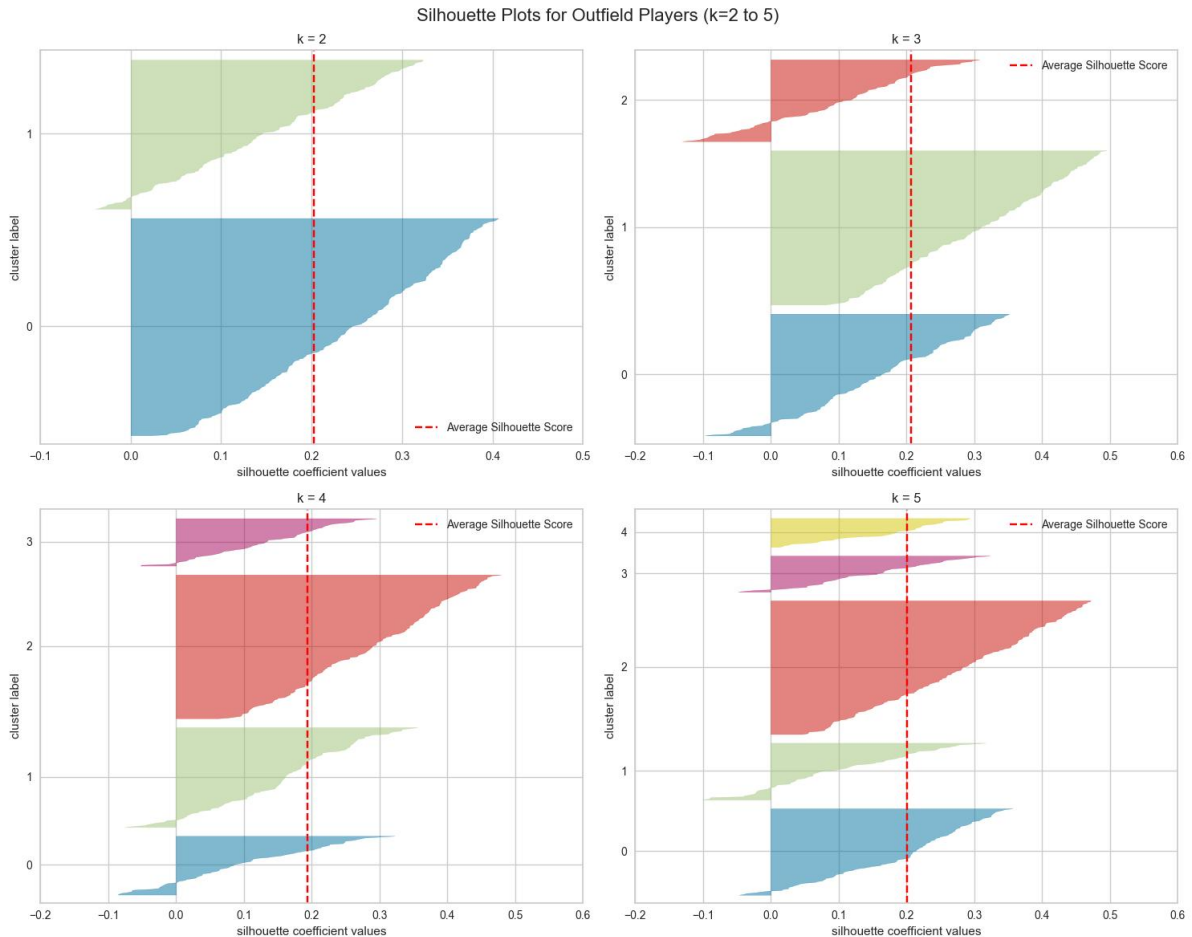
- Biểu đồ Elbow



- Đánh giá bằng Elbow Method

- Độ phân tán của mô hình giảm mạnh nhất trong giai đoạn từ $k = 2$ đến $k = 3$.
- Bắt đầu từ vị trí $k = 3$ trở đi, đồ thị bắt đầu tạo thành một góc gãy (khủy tay) và độ dốc thoải dần. Việc tăng thêm số lượng cụm ($k = 4, 5...$) chỉ giúp giảm nhẹ độ phân tán, không đáng kể so với chi phí tính toán và làm tăng độ phức tạp không cần thiết của mô hình.
- Dấu hiệu hình học này là cơ sở đầu tiên để cân nhắc chọn $k = 3$ làm điểm tối ưu.

- Biểu đồ Silhouette



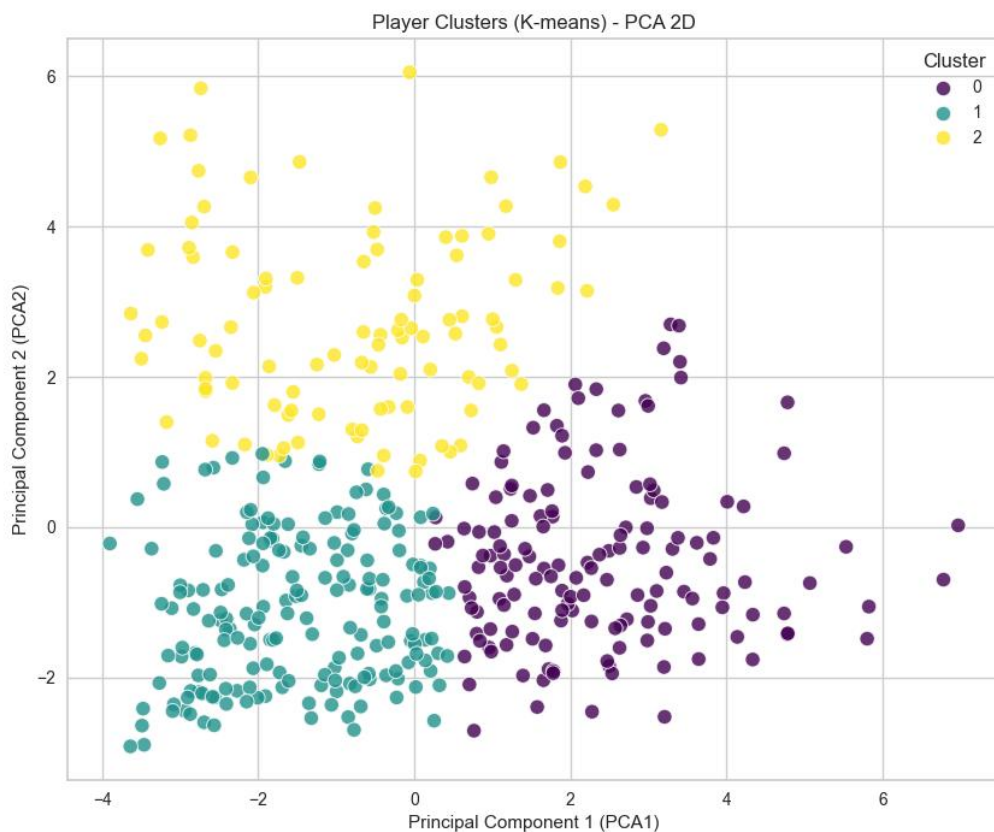
- Đánh giá bằng Hệ số Silhouette

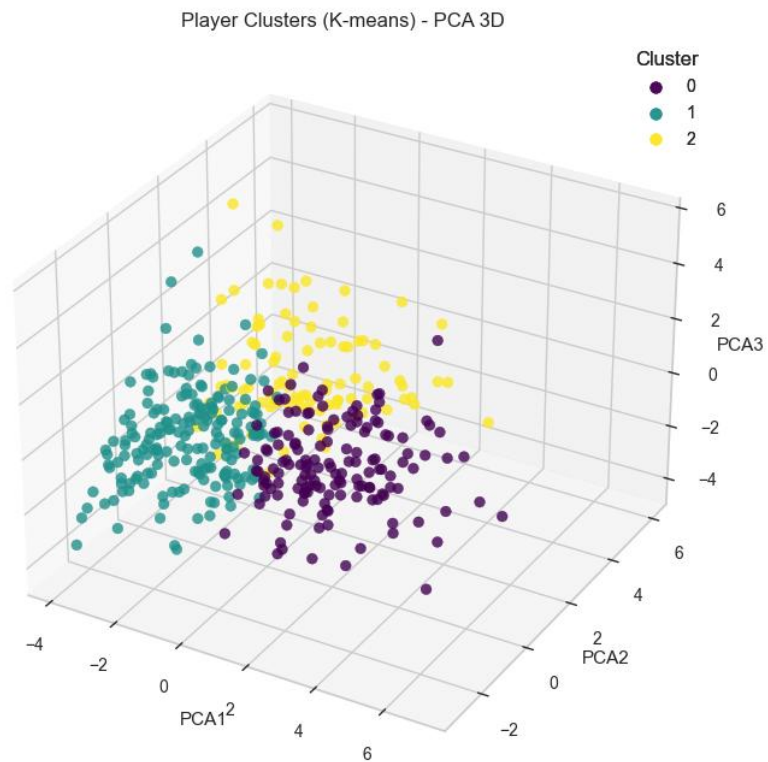
- **Tại $k = 2$:** Dù các cụm có độ dày lớn, nhưng việc chỉ chia cầu thủ tuyến trên thành 2 nhóm là quá khái quát và không phản ánh đúng thực tế chiến thuật.
- **Tại $k = 4$ và $k = 5$:** Bắt đầu xuất hiện hiện tượng "phân mảnh". Đồ thị cho thấy có những cụm quá mỏng (ít phần tử) và một lượng lớn điểm dữ liệu có hệ số Silhouette âm (bị mô hình phân loại nhầm hoặc nằm ở vùng ranh giới quá mờ nhạt giữa các cụm).
- **Tại $k = 3$:** Hệ số Silhouette trung bình (đường đứt nét màu đỏ) duy trì ở mức ổn định (~ 0.2). Mặc dù vẫn tồn tại một số điểm dữ liệu mang giá trị âm ở cụm 0 và cụm 2 (thể hiện sự kết hợp về mặt chỉ số của những cầu thủ như "hậu vệ cánh dâng cao" hay "tiền vệ công"), nhưng nhìn chung các phần lớn dữ liệu của cả 3 cụm đều vượt qua ngưỡng trung bình. Phân bố độ dày (số lượng cầu thủ) giữa các cụm cũng tương đối hài hòa và hợp lý nhất so với các trường hợp khác.

=> **Kết luận:** Giá trị **k = 3** là lựa chọn hoàn hảo và logic nhất. Con số này phản ánh chính xác 3 tuyến chiến thuật cơ bản của các cầu thủ ngoài thủ môn (Outfield Players) trên sân cỏ, bao gồm:

- **Nhóm 1 (Trụ cột phòng ngự và Phân phối bóng):** Các cầu thủ có thời gian thi đấu cao cùng các chỉ số phòng ngự và hỗ trợ lối chơi nổi bật như Tắc bóng, Cắt bóng và Tạt bóng (hậu vệ, hậu vệ cánh và tiền vệ trung tâm/phòng ngự).
- **Nhóm 2 (Xoay tua & Dự bị):** Các cầu thủ có thời gian thi đấu thấp nên hầu hết các thông số tích lũy đều ở mức nhỏ (cầu thủ dự bị, cầu thủ trẻ, cầu thủ nghỉ thi đấu dài hạn do chấn thương).
- **Nhóm 3 (Tấn công):** Các cầu thủ sở hữu các chỉ số tấn công vượt trội như Số cú sút mỗi 90 phút và Bàn thắng mỗi 90 phút (tiền đạo cắm, tiền đạo cánh và tiền vệ tấn công).

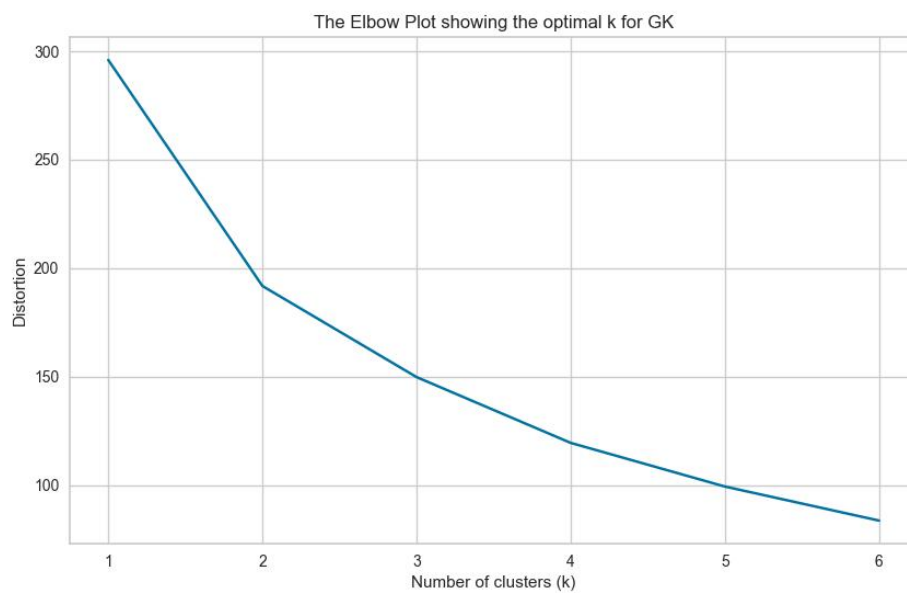
- Sử dụng thuật toán PCA vẽ scatter plot phân cụm dữ liệu trên mặt 2D, 3D (Cluster 0 : Nhóm 1, Cluster 1 : Nhóm 2, Cluster 2 : Nhóm 3)





5.3.2 Phân cụm đối với thủ môn (Goalkeepers)

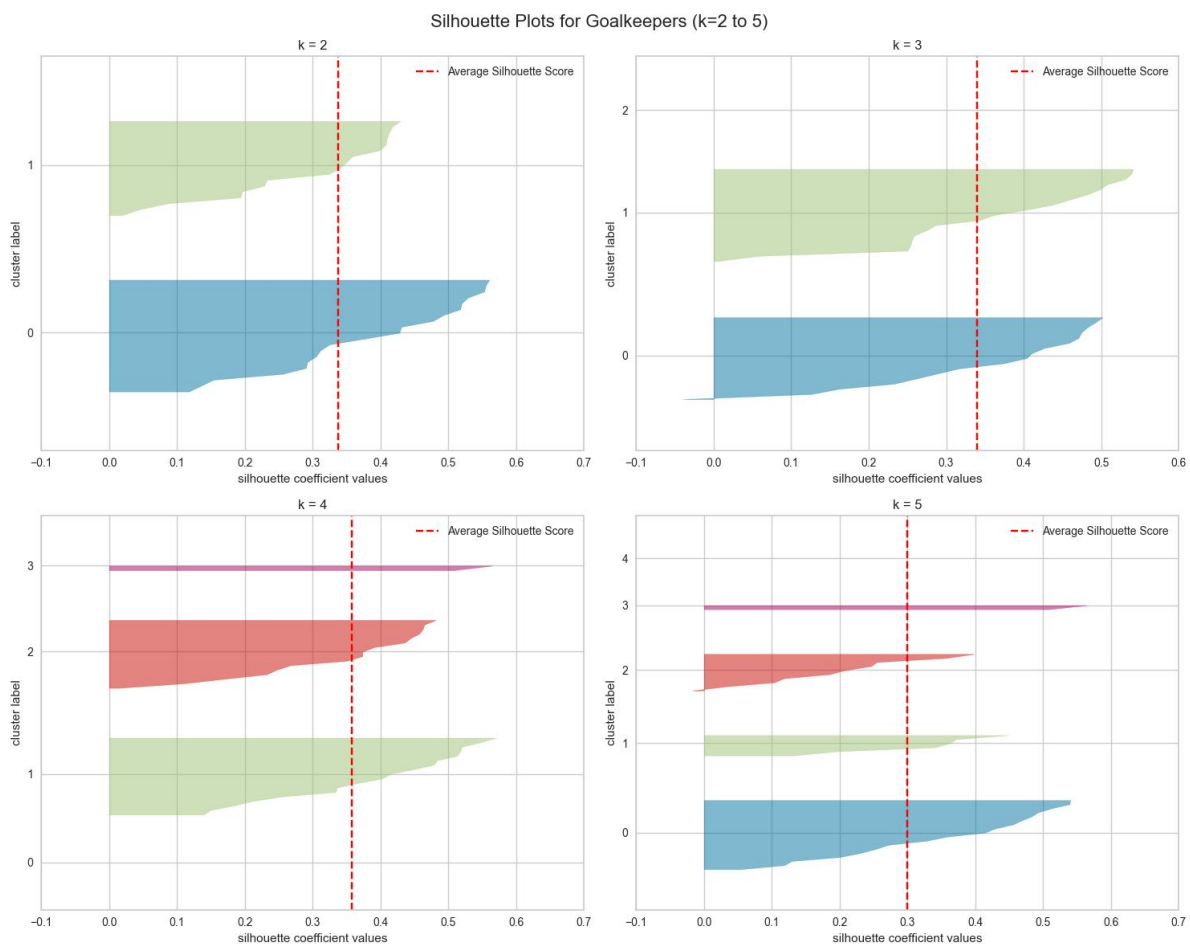
- Biểu đồ Elbow



- Đánh giá bằng Elbow Method

- Độ phân tán (Distortion) sụt giảm cực kỳ mạnh mẽ khi chuyển từ $k = 1$ sang $k = 2$.
- Ngay tại vị trí $k = 2$, đồ thị tạo thành một điểm gãy khúc (khuyết tay) rất rõ nét. Từ $k = 3$ trở đi, đường cong trở nên thoải dần và độ sụt giảm không còn đáng kể. Điều này cho thấy việc chia thành 2 cụm đã bao quát được phần lớn sự khác biệt trong tập dữ liệu.

- Biểu đồ Silhouette



- Đánh giá bằng Hệ số Silhouette

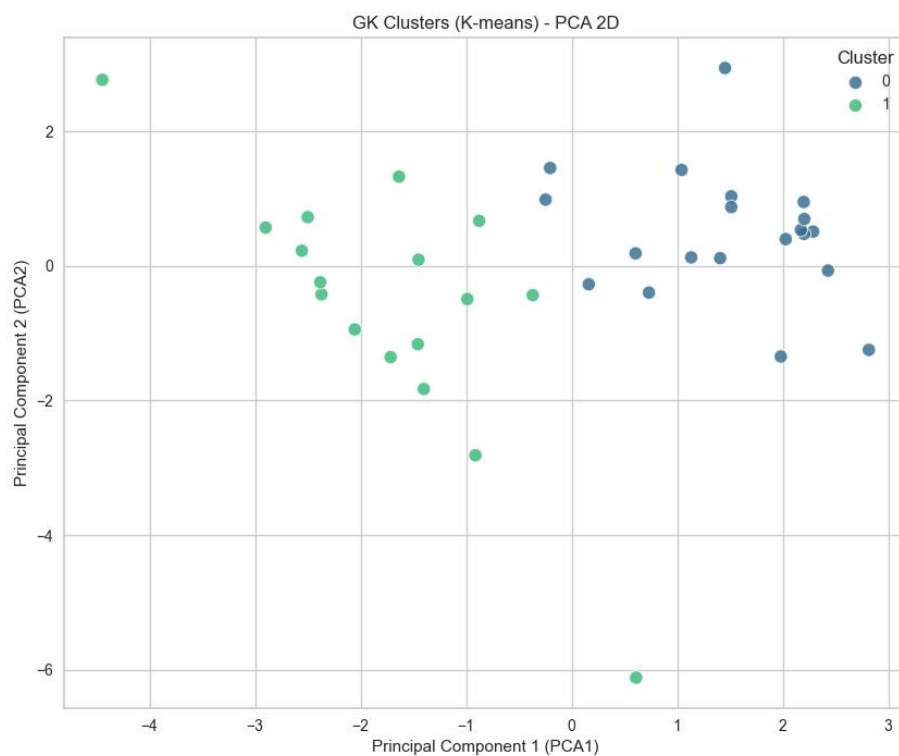
- **Tại $k = 2$:** Hệ số Silhouette trung bình (đường đứt nét màu đỏ) đạt mức rất cao (vượt ngưỡng 0.5). Cả hai cụm đều có độ kết dính tốt, hình dáng các cụm đồng đều và **hoàn toàn không có** dữ liệu bị âm (phân loại sai). Đây là trạng thái phân cụm lý tưởng nhất.
- **Tại $k = 3, 4, 5$:** Điểm trung bình Silhouette sụt giảm nghiêm trọng. Đồ thị bắt đầu xuất hiện hiện tượng "phân mảnh" với những cụm rất mỏng và

hàng loạt các điểm dữ liệu mang giá trị âm, chứng tỏ việc cố tình chia thành nhiều hơn 2 nhóm sẽ phá vỡ cấu trúc tự nhiên của dữ liệu.

=> **Kết luận:** Giá trị $k = 2$ là lựa chọn hoàn hảo và logic nhất. Con số này phản ánh chính xác 2 cấp độ phân hóa cơ bản của vị trí Thủ môn (Goalkeepers) trong đội hình, bao gồm:

- **Nhóm 1 (Thủ môn bắt chính):** Các cầu thủ có thời gian thi đấu cao nhất cùng các thông số chuyên môn vượt trội như Số pha cứu thua, Số trận giữ sạch lưới và Số trận thắng (thủ môn số 1 của câu lạc bộ).
- **Nhóm 2 (Thủ môn dự bị):** Các cầu thủ có thời gian thi đấu rất thấp nên hầu hết các thông số tích lũy đều ở mức tối thiểu (thủ môn số 2, số 3, thủ môn trẻ dự bị, thủ môn nghỉ thi đấu dài hạn do chấn thương).

- **Sử dụng thuật toán PCA vẽ scatter plot phân cụm dữ liệu trên mặt 2D, 3D**
(Cluster 0 : Nhóm 1, Cluster 1 : Nhóm 2)



GK Clusters (K-means) - PCA 3D

